



المحاضرة الخامسة:

الحلقات والملفات

Loops and Files

Topics for Lecture 5

- 5.1 The Increment and Decrement Operators
- 5.2 Introduction to Loops: The while Loop
- 5.3 Using the while Loop for Input Validation
- 5.4 Counters
- 5.5 The do-while Loop
- 5.6 The for Loop
- 5.7 Keeping a Running Total
- 5.8 Sentinels
- 5.9 Focus on Software Engineering: Deciding Which Loop to Use
- 5.10 Nested Loops
- 5.11 Using Files for Data Storage
- 5.12 Optional Topics: Breaking and Continuing a Loop

5.1 The Increment and Decrement Operators

المفهوم: ++ و -- هي عوامل تُضيف وتطرح الرقم 1 من المعاملات الخاصة بها.

<i>Increment ++</i>	<i>Decrement --</i>
num = num + 1; num += 1;	num = num - 1; num -= 1;
<i>postfix mode</i>	<i>postfix mode</i>
num++;	num--;
<i>Prefix mode</i>	<i>Prefix mode</i>
++num;	--num;

في كلا الوضعين اللاحق (postfix) والسابق (prefix)، تضيف هذه العوامل الرقم 1 أو تطرحه من المعامل الخاص بها. يوضح البرنامج 5-1 كيفية عملها.

Program 5-1

```
1 // This program demonstrates the ++ and -- operators.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int num = 4; // num starts out with 4.
8
9     // Display the value in num.
10    cout << "The variable num is " << num << endl;
11    cout << "I will now increment num.\n\n";
12
13    // Use postfix ++ to increment num.
14    num++;
15    cout << "Now the variable num is " << num << endl;
16    cout << "I will increment num again.\n\n";
17
18    // Use prefix ++ to increment num.
19    ++num;
20    cout << "Now the variable num is " << num << endl;
21    cout << "I will now decrement num.\n\n";
22
23    // Use postfix -- to decrement num.
24    num--;
25    cout << "Now the variable num is " << num << endl;
26    cout << "I will decrement num again.\n\n";
27
28    // Use prefix -- to decrement num.
29    --num;
30    cout << "Now the variable num is " << num << endl;
31    return 0;
32 }
```

Program Output

```
The variable num is 4
I will now increment num.
```

Now the variable num is 5
I will increment num again.

Now the variable num is 6
I will now decrement num.

Now the variable num is 5
I will decrement num again.

Now the variable num is 4

The Difference between Postfix and Prefix Modes

الفرق يصبح هامًا عند استخدام هذه العوامل في جمل تقوم بأكثر من مجرد عملية زيادة أو نقصان.
خذ مثالًا العبارات الآتية:

<i>postfix mode</i>	<i>Prefix mode</i>
num = 4; cout << num++;	num = 4; cout << ++num;
Output: 4 (but num has a value 5)	Output: 5 (num has a value 5)

جملة 'cout' هذه تقوم بأمرين:

(1) عرض قيمة المتغير 'num'،

(2) زيادة قيمة 'num'.

لكن أي منهما يحدث أولاً؟ سيُظهر 'cout' قيمة مختلفة إذا زادت 'num' أولاً مقارنةً بزيادتها أخيراً.
الإجابة تعتمد على وضعية (نمط) عامل الزيادة. ستعرض الناتج بشكل مختلف إذا تمت الزيادة قبل العرض عما إذا تمت بعده. هذا يعتمد على ما إذا كان العامل مُسبقاً (prefix) أو لاحقاً (postfix).

يوضح البرنامج 5-2 كيفية عملها.

Program 5-2

```
1 // This program demonstrates the prefix and postfix
2 // modes of the increment and decrement operators.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int num = 4;
9
10    cout << num << endl;    // Displays 4
11    cout << num++ << endl;  // Displays 4, then adds 1 to num
12    cout << num << endl;    // Displays 5
13    cout << ++num << endl;  // Adds 1 to num, then displays 6
14    cout << endl;          // Displays a blank line
15
```

Program 5-2

(continued)

```

16      cout << num << endl;    // Displays 6
17      cout << num-- << endl; // Displays 6, then subtracts 1 from num
18      cout << num << endl;    // Displays 5
19      cout << --num << endl; // Subtracts 1 from num, then displays 4
20
21      return 0;
22  }

```

Program Output

```

4
4
5
6

6
6
5
4

```

لنأخذ مثالاً آخر، انظر إلى الكود التالي:

```
int x = 1;
```

```
int y;
```

```
y = x++; // Postfix increment
```

- العبارة الأولى تُعرف المتغير 'x' (بقيمة ابتدائية تساوي 1)،

- والعبارة الثانية تُعرف المتغير 'y'.

- أما العبارة الثالثة فنقوم بأمرين:

• تُعَيِّن قيمة 'x' للمتغير 'y'.

• ثم تزيد قيمة 'x' بمقدار 1.

القيمة التي سيتم تخزينها في 'y' تعتمد على توقيت عملية الزيادة. وبما أن عامل '++' مستخدم هنا بالوضع اللاحق (postfix)، فإنه يعمل بعد عملية التعيين (الإسناد). لذا، سينتهي هذا الكود بتخزين القيمة 1 في 'y'. وبعد التنفيذ، ستصبح قيمة 'x' تساوي 2.

الآن لننظر إلى نفس الكود، ولكن باستخدام عامل '++' بالوضع السابق (prefix):

```
int x = 1;
```

```
int y;
```

```
y = ++x; // Prefix increment
```

هنا، سيتم زيادة 'x' قبل عملية التعيين، لذا:

- ستزيد 'x' أولاً لتصبح 2،

- ثم تُعَيِّن قيمتها الجديدة لـ 'y'.

وبالتالي، ستنتهي القيمتان بـ 'x = 2' و 'y = 2'.

ملاحظة: الوضع السابق (prefix) يعني "زيادة ثم استخدم"، بينما اللاحق (postfix) يعني "استخدم ثم زد".

Using ++ and -- in Mathematical Expressions

Example	Example
<pre>a = 2; b = 5; c = a * b++; cout << a << " " << b << " " << c;</pre>	<pre>a = 2; b = 5; c = a * ++b; cout << a << " " << b << " " << c;</pre>
The <u>cout</u> statement will display: 2 6 10	The <u>cout</u> statement will display: 2 6 12

ملاحظة: انظر إلى المثال التالي:

```
a = 2;
b = 5;
c = ++(a * b); // Error!
```

قد تعتقد أن 'c' سيتم إسناده بالقيمة 11، ولكن هذا خاطئ. لأنّ معاملات عوامل الزيادة (++) والنقصان (--) يجب أن تكون (lvalue).

Using ++ and -- in Relational Expressions

انظر المثال الآتي:

```
x = 10;
if (x++ > 10)
    cout << "x is greater than 10.\n";
```

هنا في عبارة if الشرطية تحصل عمليتين:

- مقارنة قيمة x (التي هي 10) بالعدد 10 (أولاً).
- ثم زيادة قيمة x (لتصبح 11).

وفي هذه الحالة فإن عبارة cout لن تُنفَّذ. أما إذا تمّ تغييرها بالشكل التالي فإنّ عبارة cout سيتمّ تنفيذها (لماذا برأيك؟).

```
x = 10;
if (++x > 10)
    cout << "x is greater than 10.\n";
```

5.2 Introduction to Loops: The while Loop

المفهوم: الحلقة هي جزء من البرنامج يتكرر.
الحلقة هي بنية تحكم يجعل عبارة أو مجموعة من العبارات تتكرر.
تحتوي لغة C++ على ثلاث بنى تحكم للحلقات:

1. حلقة `while`

2. حلقة `do-while`

3. حلقة `for`

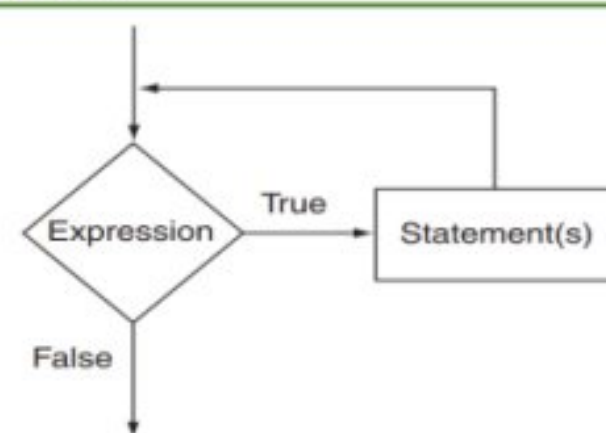
الفرق بين هذه الهياكل هو كيفية التحكم في التكرار.

شرح مبسط:

- حلقة `while`: تكرر العبارات طالما كان الشرط صحيحًا.
 - حلقة `do-while`: تنفذ العبارات مرة واحدة على الأقل، ثم تتكرر طالما كان الشرط صحيحًا.
 - حلقة `for`: تستخدم عادةً عندما يكون عدد التكرارات معروفًا مسبقًا، حيث تجمع التهيئة والشرط والتحديث في سطر واحد.
- كل نوع من الحلقات له استخداماته المناسبة حسب الحاجة في البرمجة.

The while Loop

Figure 5-1 Logic of a while loop



Here is the general format of the while loop:

```
while (expression)
    statement;
```

إليك كيف تعمل الحلقة: يتم اختبار التعبير الشرطي، وإذا كان صحيحًا (true)، يتم تنفيذ العبارة (أو مجموعة العبارات). ثم يُعاد اختبار التعبير الشرطي مرة أخرى، فإذا ظل صحيحًا، تُنفذ العبارة. هذه الدورة تتكرر حتى يصبح التعبير الشرطي خاطئًا (false)، وعندها تتوقف الحلقة.

توضيح:

1. الاختبار الأول: يتم تقييم الشرط.
 - إذا كان `true` → يُنفذ الكود داخل الحلقة.
 - إذا كان `false` → تتخطى الحلقة ولا تنفذ أي شيء.
2. بعد كل تنفيذ: يعود البرنامج لاختبار الشرط مرة أخرى.
3. التكرار: يستمر هذا (اختبار → تنفيذ) حتى يصبح الشرط `false`.

If you wish the `while` loop to repeat a block of statements, its format is:

```
while (expression)
{
    statement;
    statement;
    // Place as many statements here
    // as necessary.
}
```

The `while` loop works like an `if` statement that executes over and over. As long as the expression inside the parentheses is true, the conditionally executed statement or block will repeat. Program 5-3 uses the `while` loop to print “Hello” five times.

Program 5-3

```
1  // This program demonstrates a simple while loop.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int number = 0;
8
9      while (number < 5)
10     {
11         cout << "Hello\n";
12         number++;
13     }
14     cout << "That's all!\n";
15     return 0;
16 }
```

Program Output

```
Hello
Hello
Hello
Hello
Hello
That's all!
```

Figure 5-2 Actions of the while loop

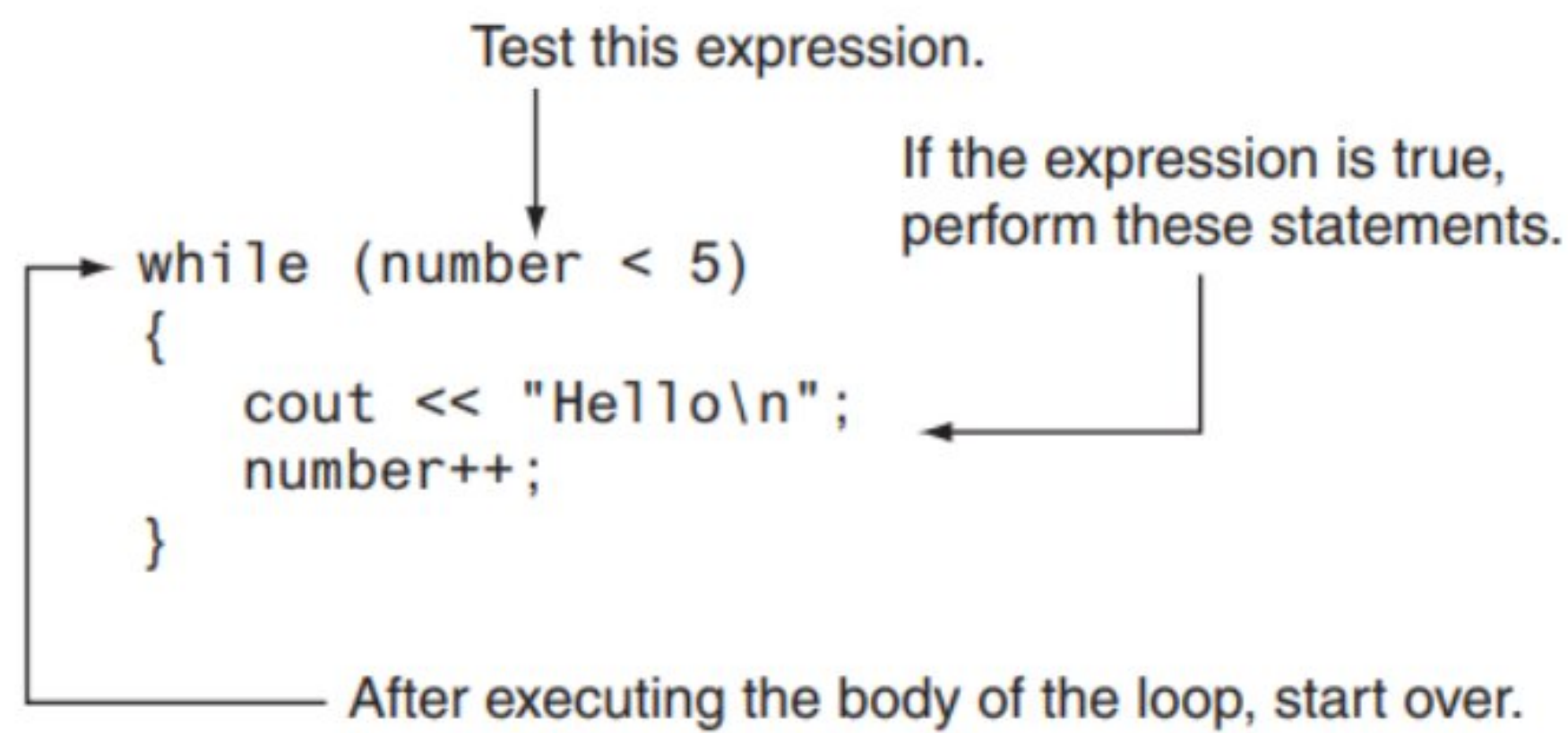
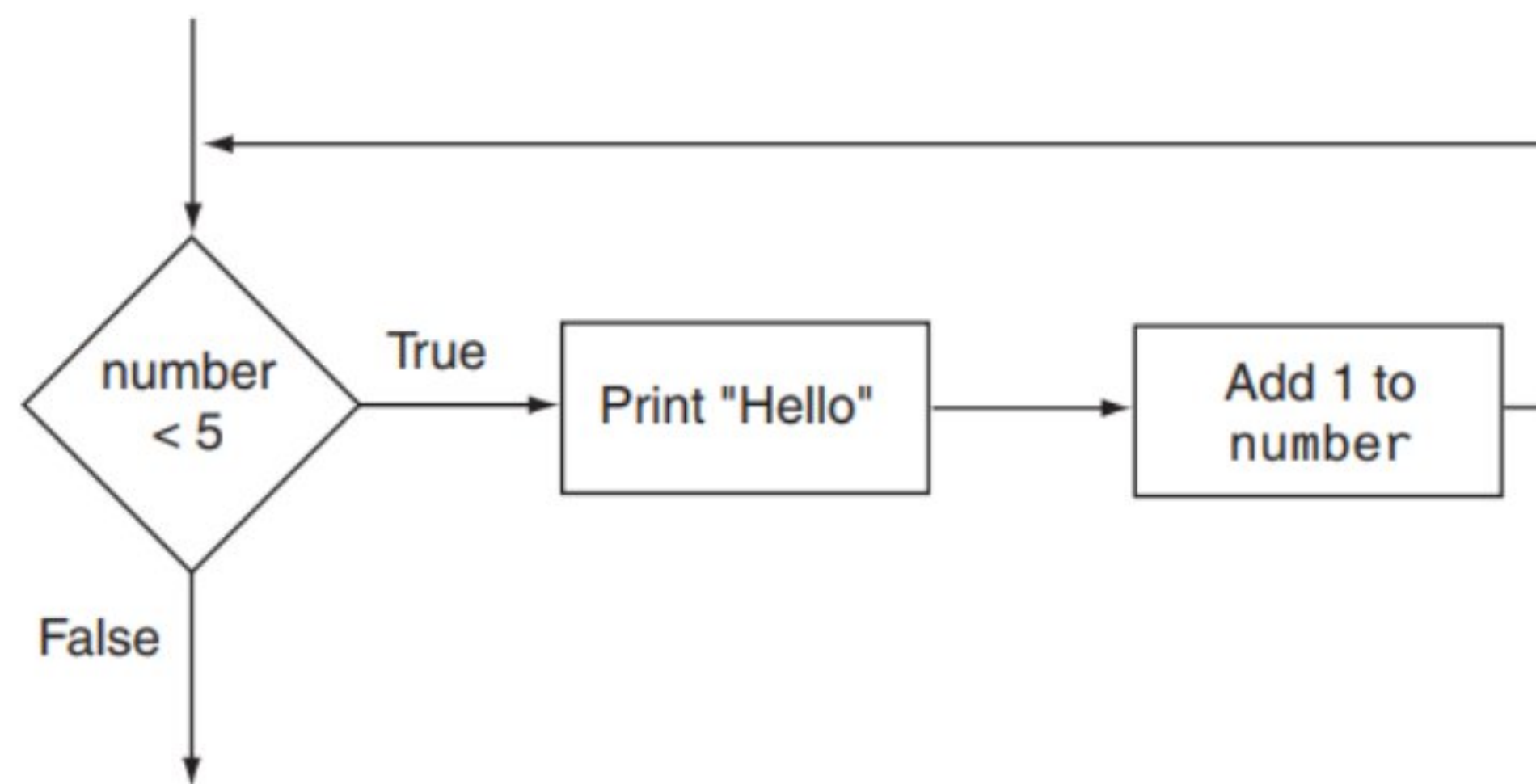


Figure 5-3 Flowchart for the while loop



The while Loop Is a Pretest Loop

تُعرف حلقة `while` باسم حلقة الاختبار المسبق (pretest loop)، مما يعني أنها تختبر تعبيرها الشرطي قبل كل تكرار.

لاحظ تعريف المتغير في السطر 7 من البرنامج 3-5:

```
int number = 0;
```

تم تهيئة المتغير `number` بالقيمة `0`. أما إذا كان قد تم تهيئته بقيمة `5` أو أكبر، كما في المقطع التالي، فلن يتم تنفيذ الحلقة أبدًا:

```
int number = 6;
```

```
while (number < 5)
```

```
{
```

```
    cout << "Hello\n";
```

```
    number++;
```

```
}
```

خاصية مهمة في حلقة `while` :

- لا تنفذ الحلقة ولو لمرة واحدة إذا كان الشرط خاطئاً من البداية.
 - إذا كنت تريد التأكد من أن الحلقة ستنفذ على الأقل مرة واحدة، فيجب عليك تهيئة البيانات ذات الصلة بحيث يكون التعبير الشرطي صحيحاً في البداية.
- مثال توضيحي:

```
int number = 0; // تم التهيئة بقيمة تجعل الشرط صحيحاً في البداية
while (number < 5)
{
    cout << "Hello\n";
    number++;
}
```

هنا، لأن `number` تساوي `0` (أي أقل من `5`)، فإن الحلقة ستنفذ وتطبع `"Hello"` حتى يصل `number` إلى `5`.

باختصار:

✓ حلقة `while` تختبر الشرط أولاً ثم تنفذ الأوامر إذا كان صحيحاً.

✗ إذا كان الشرط خاطئاً من البداية، لن يتم تنفيذ الحلقة مطلقاً.

🔧 لضمان التنفيذ، يجب أن تبدأ بقيمة تجعل الشرط صحيحاً.

Infinite Loops

Here is an example of an infinite loop:

```
int number = 0;
while (number < 5)
{
    cout << "Hello\n";
}
```

It's also possible to create an **infinite loop** by accidentally placing a semicolon after the first line of the while loop. Here is an example:

```
int number = 0;
while (number < 5); // This semicolon is an ERROR!
{
    cout << "Hello\n";
    number++;
}
```

Don't Forget the Braces with a Block of Statements

For example, look at the following code:

```
int number = 0;
// This loop is missing its braces! (Infinite loop)
while (number < 5)
    cout << "Hello\n";
    number++;
```

5.3 Using the while Loop for Input Validation

المفهوم: يمكن استخدام حلقة `while` لإنشاء إجراءات إدخال تتكرر حتى يتم إدخال بيانات مقبولة.

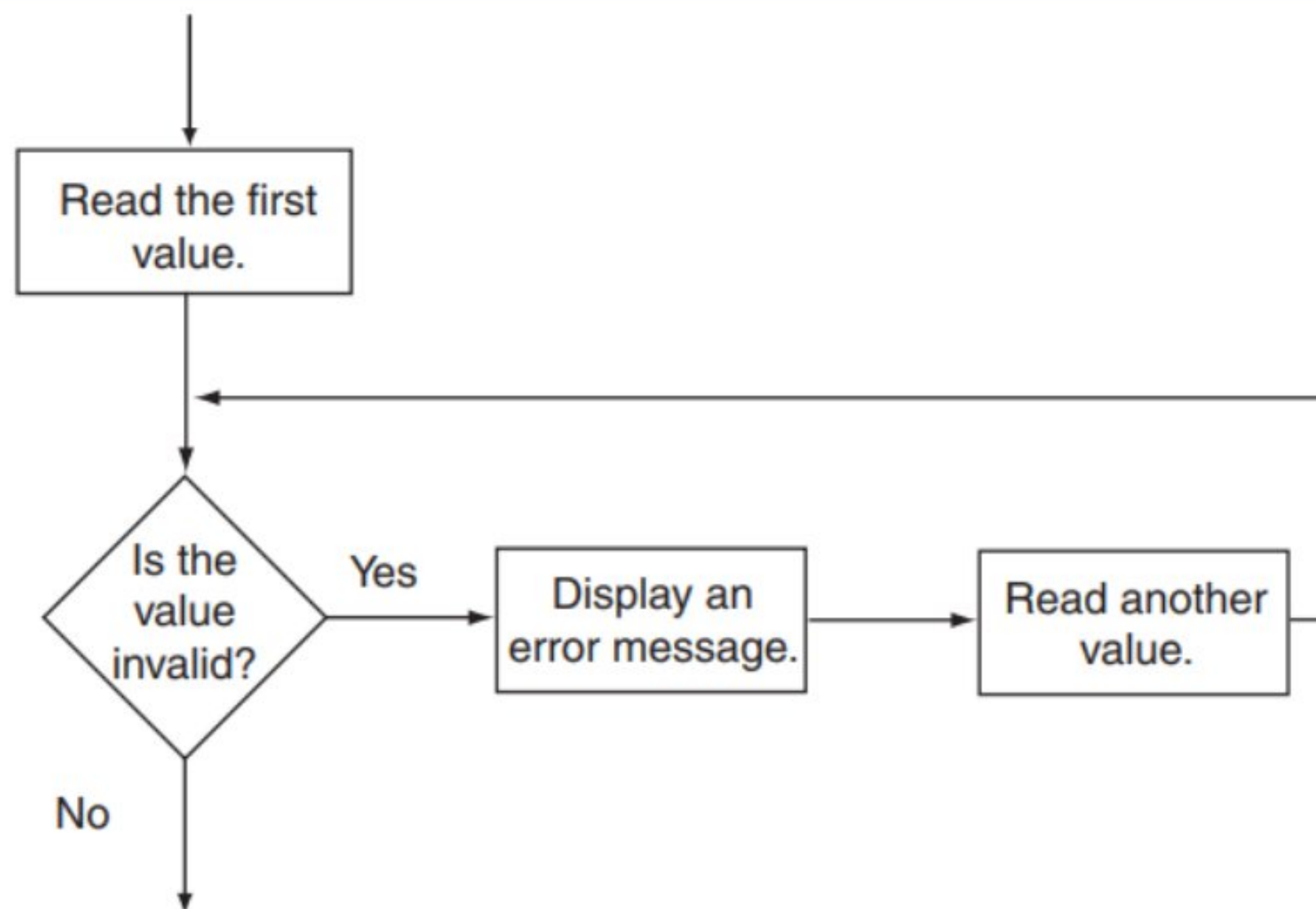
من أشهر المقولات في عالم الحوسبة:

"إذا أدخلت بيانات غير صالحة، ستكون المخرجات غير صالحة كذلك."

- موثوقية مخرجات البرنامج تعتمد على جودة مدخلاته، لذا يجب التأكد من عدم إدخال بيانات غير صالحة ("قمامة") إلى البرنامج.
 - التحقق من صحة الإدخال (Input Validation) هو عملية فحص البيانات المدخلة من قبل المستخدم وتحديد ما إذا كانت صالحة أم لا.
 - البرنامج الجيد يجب أن يقدم تعليمات واضحة حول نوع الإدخال المقبول، ولا يفترض أن المستخدم اتبع هذه التعليمات تلقائيًا.
- كيف تساعد حلقة `while` في التحقق من صحة الإدخال؟
- تعتبر حلقة `while` أداة فعالة للتحقق من صحة الإدخال، حيث يمكنها إجبار المستخدم على إعادة إدخال القيمة مرارًا وتكرارًا حتى تصبح صالحة.
- مثال: طلب رقم بين 1 و 100

```
cout << "Enter a number in the range 1-100: ";
cin >> number;
while (number < 1 || number > 100)
{
    cout << "ERROR: Enter a value in the range 1-100: ";
    cin >> number;
}
```

Figure 5-4 Flowchart for input validation



Program 5-5

```
1 // This program calculates the number of soccer teams
2 // that a youth league may create from the number of
3 // available players. Input validation is demonstrated
4 // with while loops.
5 #include <iostream>
6 using namespace std;
7
8 int main()
9 {
10     // Constants for minimum and maximum players
11     const int MIN_PLAYERS = 9,
12             MAX_PLAYERS = 15;
13
14     // Variables
15     int players, // Number of available players
16         teamPlayers, // Number of desired players per team
17         numTeams, // Number of teams
18         leftOver; // Number of players left over
19
20     // Get the number of players per team.
21     cout << "How many players do you wish per team? ";
22     cin >> teamPlayers;
23 }
```

```

24 // Validate the input.
25 while (teamPlayers < MIN_PLAYERS || teamPlayers > MAX_PLAYERS)
26 {
27     // Explain the error.
28     cout << "You should have at least " << MIN_PLAYERS
29         << " but no more than " << MAX_PLAYERS << " per team.\n";
30
31     // Get the input again.
32     cout << "How many players do you wish per team? ";
33     cin >> teamPlayers;
34 }
35
36 // Get the number of players available.
37 cout << "How many players are available? ";
38 cin >> players;
39
40 // Validate the input.
41 while (players <= 0)
42 {
43     // Get the input again.
44     cout << "Please enter 0 or greater: ";
45     cin >> players;
46 }
47
48 // Calculate the number of teams.
49 numTeams = players / teamPlayers;
50
51 // Calculate the number of leftover players.
52 leftOver = players % teamPlayers;
53
54 // Display the results.
55 cout << "There will be " << numTeams << " teams with "
56     << leftOver << " players left over.\n";
57 return 0;
58 }

```

Program Output with Example Input Shown in Bold

```

How many players do you wish per team? 4 
You should have at least 9 but no more than 15 per team.
How many players do you wish per team? 12 
How many players are available? -142 
Please enter 0 or greater: 142 
There will be 11 teams with 10 players left over.

```

5.4 Counters

المفهوم: العداد هو متغير يتم زيادته أو إنقاصه بانتظام في كل مرة تتكرر فيها الحلقة. في بعض الأحيان، من المهم للبرنامج أن يتحكم أو يتتبع عدد المرات التي تنفذ فيها الحلقة تكرارها. على سبيل المثال، يعرض البرنامج 5-6 جدولاً يتكون من الأرقام من 1 إلى 10 ومربعاتها، لذلك يجب أن تكرر حلقتها 10 مرات.

Program 5-6

```
1 // This program displays a list of numbers and
2 // their squares.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int MIN_NUMBER = 1, // Starting number to square
9         MAX_NUMBER = 10; // Maximum number to square
10
```

Program 5-6 (continued)

```
11     int num = MIN_NUMBER; // Counter
12
13     cout << "Number Number Squared\n";
14     cout << "-----\n";
15     while (num <= MAX_NUMBER)
16     {
17         cout << num << "\t\t" << (num * num) << endl;
18         num++; //Increment the counter.
19     }
20     return 0;
21 }
```

Program Output

Number Number Squared

1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

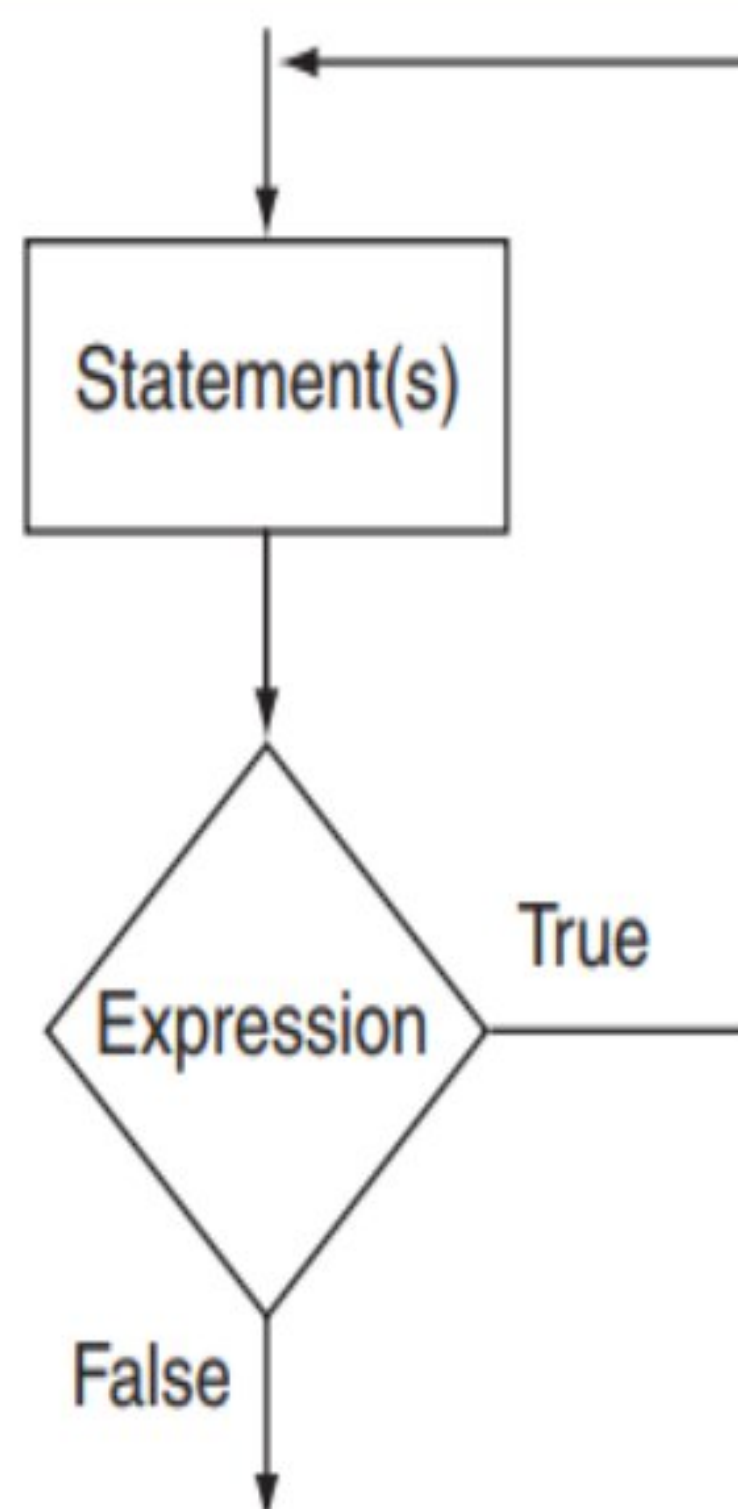
5.5 The do-while Loop

المفهوم: حلقة `do-while` هي حلقة اختبار لاحق (posttest loop)، مما يعني أن التعبير الشرطي يُختبر بعد كل تكرار.

الفرق الرئيسي بين `while` و `do-while`	
`while` (اختبار مسبق)	`do-while` (اختبار لاحق)
تختبر الشرط قبل التنفيذ	تختبر الشرط بعد التنفيذ
قد لا تنفذ أبدًا إذا كان الشرط خاطئًا من البداية	تنفذ مرة واحدة على الأقل حتى لو كان الشرط خاطئًا

Here is the do-while loop's format when the body of the loop contains only a single statement:	Here is the format of the do-while loop when the body of the loop contains multiple statements:
<pre>do statement; while (expression);</pre>	<pre>do { statement; statement; // Place as many statements here // as necessary. } while (expression);</pre>
 NOTE: The do-while loop must be terminated with a semicolon.	

Figure 5-5 Flowchart for a do-while loop



Program 5-7

```
1 // This program averages 3 test scores. It repeats as
2 // many times as the user wishes.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int score1, score2, score3; // Three scores
9     double average;             // Average score
10    char again;                  // To hold Y or N input
11
12    do
13    {
14        // Get three scores.
15        cout << "Enter 3 scores and I will average them: ";
16        cin >> score1 >> score2 >> score3;
17
18        // Calculate and display the average.
19        average = (score1 + score2 + score3) / 3.0;
20        cout << "The average is " << average << ".\n";
21
22        // Does the user want to average another set?
23        cout << "Do you want to average another set? (Y/N) ";
24        cin >> again;
25    } while (again == 'Y' || again == 'y');
26    return 0;
27 }
```

Program Output with Example Input Shown in Bold

```
Enter 3 scores and I will average them: 80 90 70 
The average is 80.
Do you want to average another set? (Y/N) y 
Enter 3 scores and I will average them: 60 75 88 
The average is 74.3333.
Do you want to average another set? (Y/N) n 
```

5.6 The for Loop

المفهوم: حلقة for مثالية لأداء عدد معروف من التكرارات. بشكل عام، هناك فئتان من الحلقات: الحلقات الشرطية والحلقات المُتحكم بها بالعدد. تُنفَّذ الحلقة الشرطية طالما وُجد شرط مُعين. على سبيل المثال، تُنفَّذ حلقة التحقق من صحة الإدخال طالما كانت قيمة الإدخال غير صالحة. عند كتابة حلقة شرطية، لا يُمكنك معرفة عدد مرات تكرارها.

أحياناً، تعرف العدد الدقيق للتكرارات التي يجب أن تُنفَّذها الحلقة. تُعرف الحلقة التي تُكرَّر عدداً مُحدداً من المرات باسم الحلقة المُتحكم بها بالعد. على سبيل المثال، إذا طلبت حلقة من المستخدم إدخال مبالغ المبيعات لكل شهر في السنة، فستُكرَّر 12 مرة. في جوهرها، تُعدّ الحلقة إلى 12 وتطلب من المستخدم إدخال مبلغ المبيعات في كل مرة تُجري فيها عملية عد. يجب أن تحتوي الحلقة المُتحكم بها بالعد على ثلاثة عناصر:

1. يجب أن تُهيئ مُتغير العداد إلى قيمة ابتدائية.
 2. يجب أن تختبر مُتغير العداد بمقارنته بقيمة قصوى. عندما يصل متغير العداد إلى أقصى قيمة له، تنتهي الحلقة.
 3. يجب تحديث متغير العداد خلال كل تكرار. يتم ذلك عادةً بزيادة قيمة المتغير.
- تُعدّ حلقات التحكم بالعداد شائعة جداً لدرجة أن لغة C++ توفر نوعاً من الحلقات مخصصاً لها. يُعرف هذا النوع باسم حلقة `for`. صُممت حلقة `for` خصيصاً لتهيئة متغير العداد واختباره وتحديثه. إليك صيغة حلقة `for` عند استخدامها لتكرار عبارة واحدة:

```
for (initialization; test; update)
    statement;
```

The format of the for loop when it is used to repeat a block is

```
for (initialization; test; update)
{
    statement;
    statement;
    // Place as many statements here
    // as necessary.
}
```

فيما يلي مثال لحلقة for بسيطة تطبع كلمة "Hello" خمس مرات:

```
for (count = 0; count < 5; count++)  
    cout << "Hello" << endl;
```

Figure 5-6 Sequence of events in a for loop

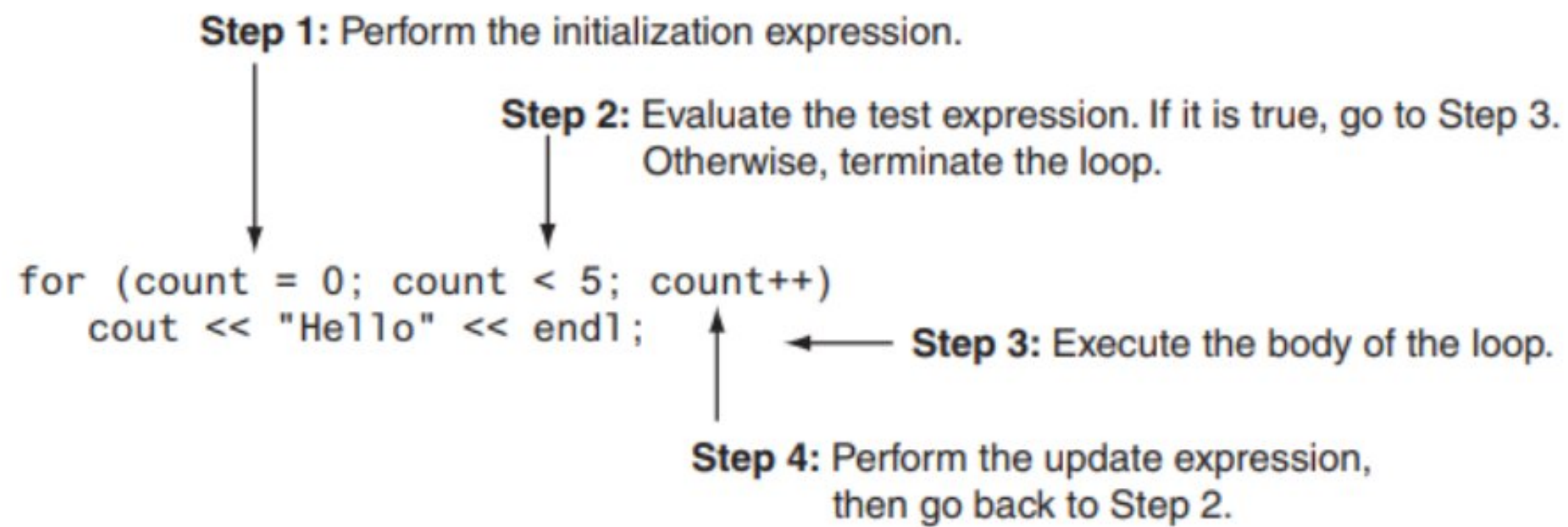
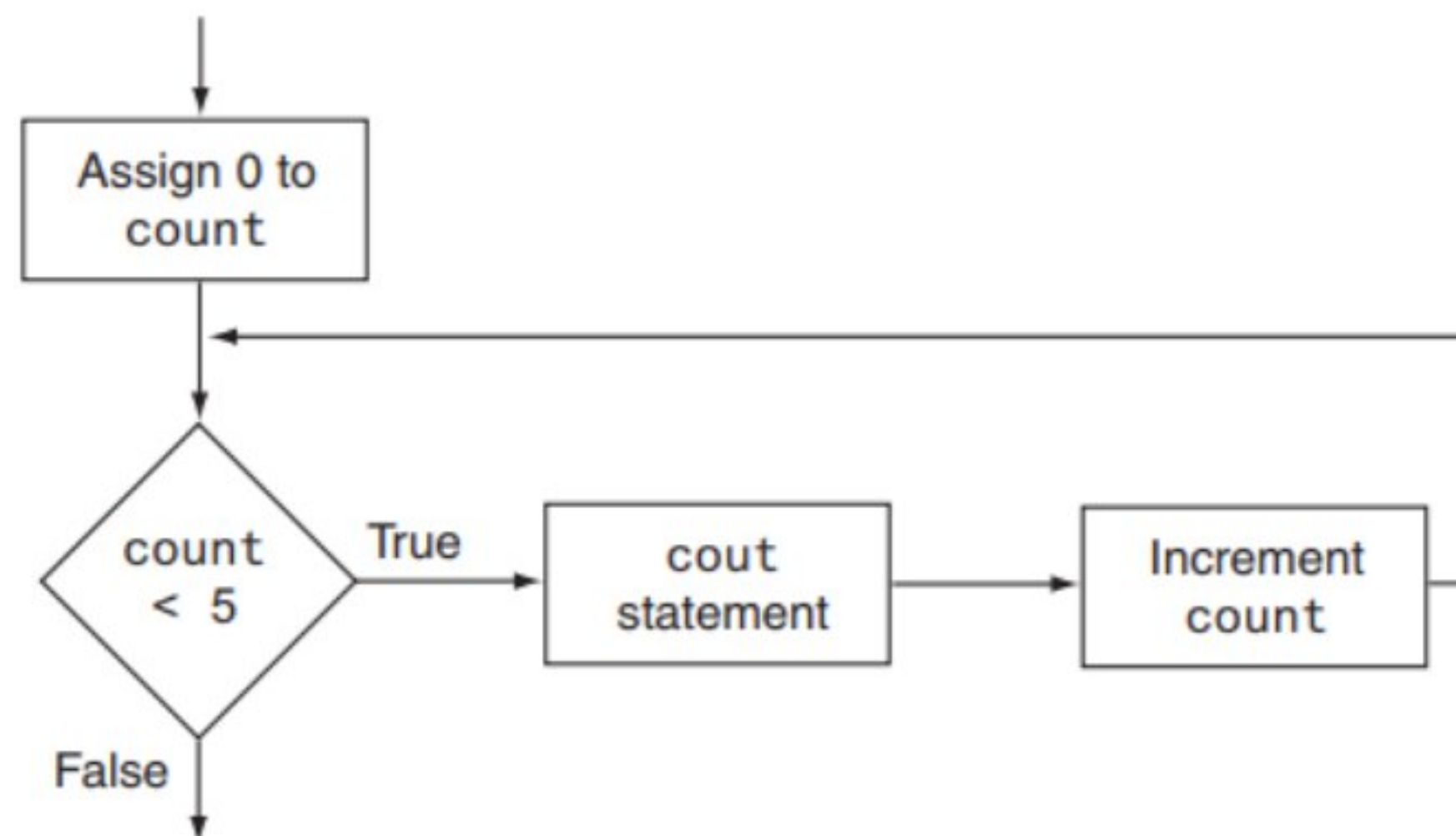


Figure 5-7 shows the loop's logic in the form of a flowchart.

Figure 5-7 for loop logic



يُظهر البرنامج 5-9 مثالاً آخر لحلقة for تستخدم متغير العداد الخاص بها داخل جسم الحلقة. هذا برنامج آخر يعرض جدولاً يُظهر الأرقام من 1 إلى 10 ومربعاتها.

Program 5-9

```
1 // This program displays the numbers 1 through 10 and
2 // their squares.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int MIN_NUMBER = 1,    // Starting value
9           MAX_NUMBER = 10;    // Ending value
10    int num;
11
12    cout << "Number Number Squared\n";
13    cout << "-----\n";
14
15    for (num = MIN_NUMBER; num <= MAX_NUMBER; num++)
16        cout << num << "\t\t" << (num * num) << endl;
17
18    return 0;
19 }
```

Program Output

```
Number Number Squared
-----
1           1
2           4
3           9
4          16
5          25
6          36
7          49
8          64
9          81
10         100
```

يوضح الشكل 8-5 تسلسل الأحداث التي تنفذها حلقة for هذه، ويوضح الشكل 9-5 منطق الحلقة على شكل مخطط انسيابي.

Figure 5-8 Sequence of events in the for loop

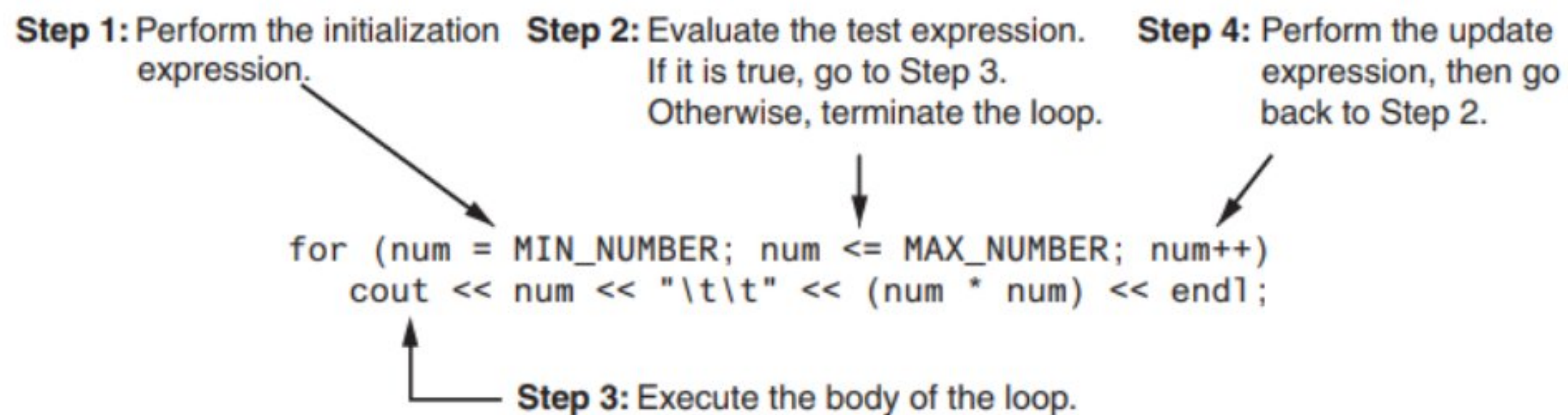
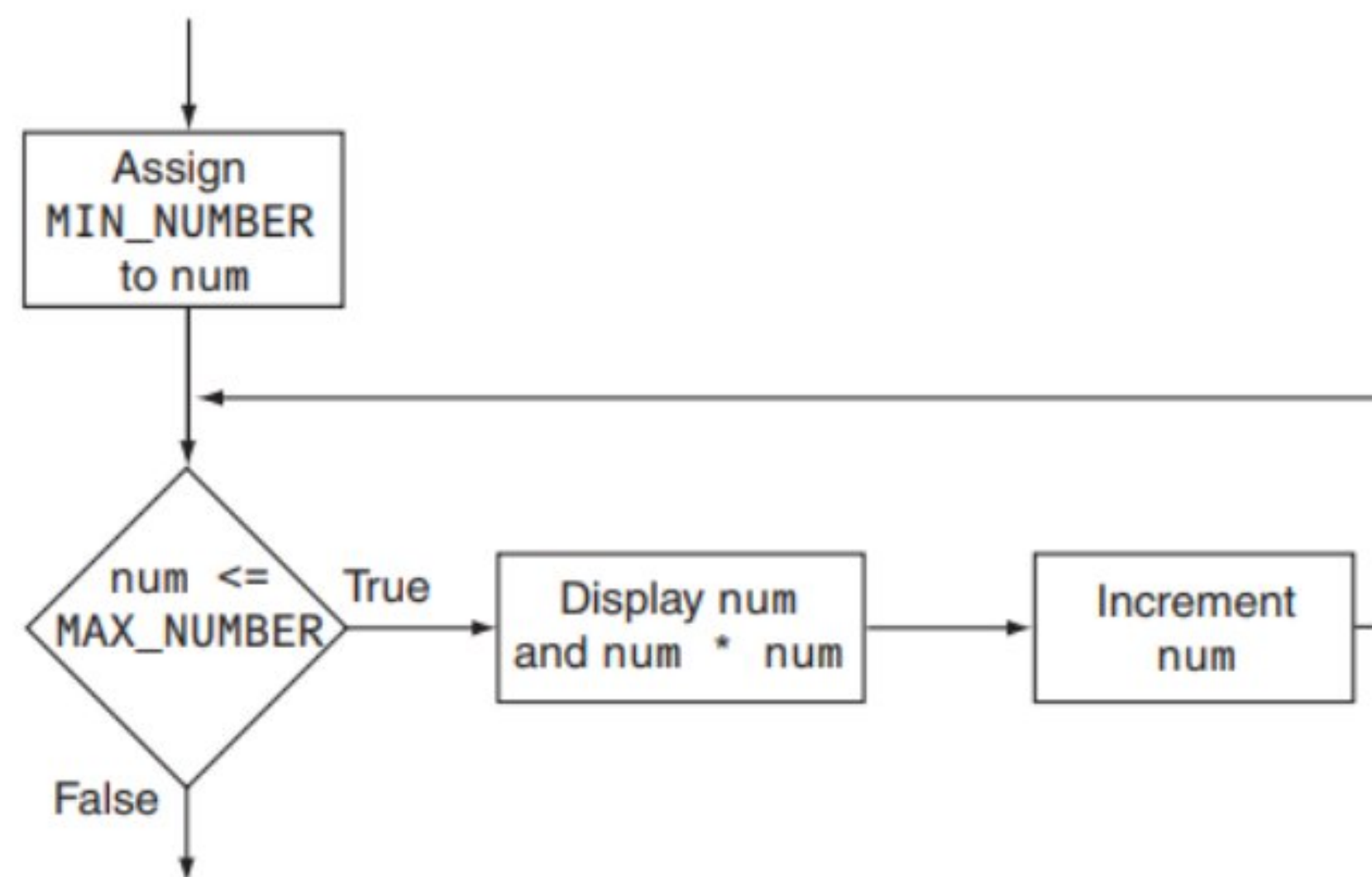
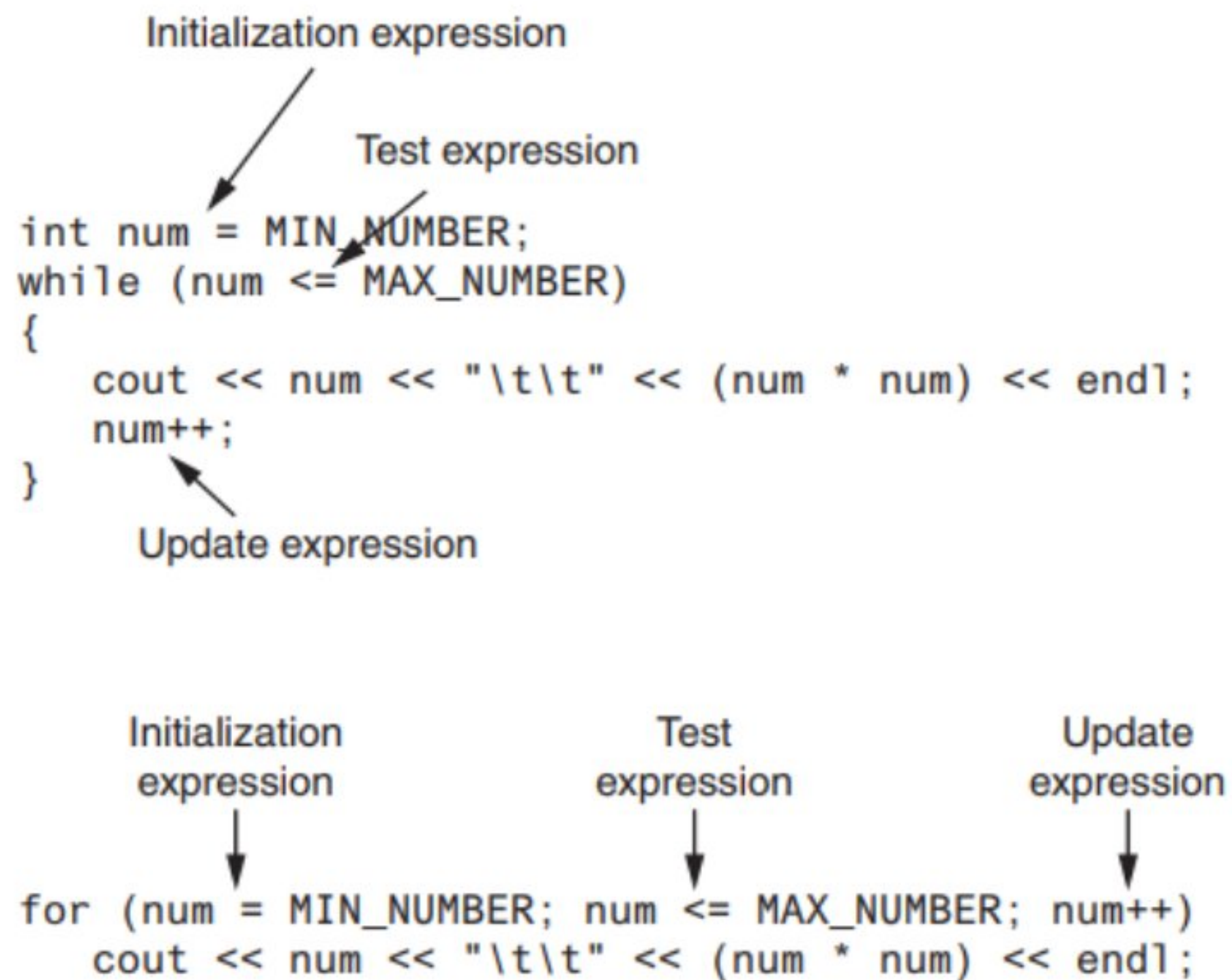


Figure 5-9 Flowchart for the for loop



Using the for Loop instead of while or do-while

Figure 5-10 Initialization, test, and update expressions



The for Loop Is a Pretest Loop

لأن حلقة `for` تختبر تعبير الاختبار الخاص بها قبل إجراء أي تكرار، فهي حلقة اختبار مسبق. من الممكن كتابة حلقة `for` بطريقة تمنع تكرارها أبدًا. إليك مثال:

```
for (count = 11; count <= 10; count++)
    cout << "Hello" << endl;
```

نظرًا لأن قيمة المتغير `count` مُهيأة لقيمة تجعل تعبير الاختبار خاطئًا من البداية، تنتهي هذه الحلقة بمجرد بدئها.

Avoid Modifying the Counter Variable in the Body of the for Loop

احرص على عدم وضع عبارة تُعدّل متغير العداد في نص حلقة `for`. يجب أن تُجرى جميع تعديلات متغير العداد في تعبير التحديث، الذي يُنفذ تلقائيًا في نهاية كل تكرار. إذا عدّلت عبارة في نص الحلقة متغير العداد أيضًا، فمن المرجّح ألا تنتهي الحلقة في الوقت المُتوقّع. على سبيل المثال، تُزيد الحلقة التالية قيمة `x` مرتين لكل تكرار:

```
for (x = 1; x <= 10; x++)
{
    cout << x << endl;
    x++; // Error!
}
```

Other Forms of the Update Expression

لست مُقيِّداً باستخدام عبارات الزيادة في تعبير التحديث. إليك حلقة عدّ تعرض جميع الأعداد الزوجية من 2 إلى 100 بإضافة 2 إلى عدّادها:

```
for (num = 2; num <= 100; num += 2)
    cout << num << endl;
```

وإليك حلقة عدّ تنازلياً من 10 إلى 0:

```
for (num = 10; num >= 0; num--)
    cout << num << endl;
```

Defining a Variable in the for-Loop's Initialization Expression

عند تعريف متغير في تعبير تهيئة حلقة for، يقتصر نطاق المتغير على الحلقة نفسها. هذا يعني أنه لا يمكنك الوصول إلى المتغير في عبارات خارج الحلقة. على سبيل المثال، لن يتم ترجمة مقطع البرنامج التالي لأن آخر عبارة cout لا يمكنها الوصول إلى المتغير count.

```
for (int count = 1; count <= 10; count++)
    cout << count << endl;
cout << "count is now " << count << endl; // ERROR!
```

Creating a User-Controlled for Loop

أحياناً تريد من المستخدم تحديد القيمة القصوى لمتغير العداد في حلقة for، وبالتالي تحديد عدد مرات تكرار الحلقة. على سبيل المثال، انظر إلى البرنامج 5-10. هذا برنامج آخر يعرض قائمة بالأرقام ومربعاتها. بدلاً من عرض الأرقام من 1 إلى 10، يسمح هذا البرنامج للمستخدم بإدخال القيم الدنيا والقصوى لعرضها.

Program 5-10

```
1 // This program demonstrates a user-controlled for loop.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int minNumber, // Starting number to square
8         maxNumber; // Maximum number to square
9
10    // Get the minimum and maximum values to display.
11    cout << "I will display a table of numbers and "
12         << "their squares.\n"
13         << "Enter the starting number: ";
14    cin >> minNumber;
15    cout << "Enter the ending number: ";
16    cin >> maxNumber;
17
18    // Display the table.
19    cout << "Number Number Squared\n"
20         << "-----\n";
21
22    for (int num = minNumber; num <= maxNumber; num++)
23        cout << num << "\t\t" << (num * num) << endl;
24
25    return 0;
26 }
```

Program Output with Example Input Shown in Bold

I will display a table of numbers and their squares.

Enter the starting number: **6**

Enter the ending number: **12**

Number Number Squared

6	36
7	49
8	64
9	81
10	100
11	121
12	144

Using Multiple Statements in the Initialization and Update Expressions

من الممكن تنفيذ أكثر من جملة في تعبير التهيئة وتعبير التحديث. عند استخدام جملة متعددة في أيٍّ من هذين التعبيرين، افصل الجمل بفاصلة. على سبيل المثال، انظر إلى الحلقة في الكود التالي، حيث تحتوي على جملتين في تعبير التهيئة:

```
int x, y;  
for (x = 1, y = 1; x <= 5; x++)  
{  
    cout << x << " plus " << y  
        << " equals " << (x + y)  
        << endl;  
}
```

This loop's initialization expression is

x = 1, y = 1

This initializes two variables, x and y. The output produced by this loop is:

```
1 plus 1 equals 2  
2 plus 1 equals 3  
3 plus 1 equals 4  
4 plus 1 equals 5  
5 plus 1 equals 6
```

يمكننا تعديل الحلقة لتنفيذ عبارتين في تعبير التحديث. إليك مثال:

```
int x, y;  
for (x = 1, y = 1; x <= 5; x++, y++)  
{  
    cout << x << " plus " << y  
        << " equals " << (x + y)  
        << endl;  
}
```

The loop's update expression is:

x++, y++

This update expression increments both the x and y variables. The output produced by this loop is:

1 plus 1 equals 2
 2 plus 2 equals 4
 3 plus 3 equals 6
 4 plus 4 equals 8
 5 plus 5 equals 10

ملاحظة: يُعد ربط عبارات متعددة بفواصل أمرًا جيدًا في تعابير التهيئة والتحديث، ولكن لا تحاول ربط تعبيرات متعددة بهذه الطريقة في تعبير الاختبار. إذا كنت ترغب في دمج تعبيرات متعددة في تعبير الاختبار، فيجب عليك استخدام عاملي && أو ||.

Omitting the for-Loop's Expressions

يمكن حذف تعبير التهيئة من داخل أقواس حلقة for إذا تم تنفيذه مسبقًا أو لم تكن هناك حاجة للتهيئة. فيما يلي مثال على الحلقة في البرنامج 5-10 مع تنفيذ التهيئة قبل الحلقة:

```
int num = 1;
for ( ; num <= maxVal; num++)
    cout << num << "\t\t" << (num * num) << endl;
```

يمكنك أيضًا حذف تعبير التحديث إذا كان يُنفَّذ في مكان آخر من الحلقة، أو إذا لم تكن هناك حاجة إليه. مع أن هذا النوع من التعليمات البرمجية غير موصى به، إلا أن حلقة for التالية تعمل تمامًا مثل حلقة while:

```
int num = 1;
for ( ; num <= maxVal; )
{
    cout << num << "\t\t" << (num * num) << endl;
    num++;
}
```

يمكنك حتى حذف جميع التعبيرات الثلاثة من أقواس حلقة for. مع ذلك، انتبه، إذا حذفت تعبير الاختبار، فلن يكون للحلقة طريقة مُدمجة للإنتهاء. إليك مثال:

```
for ( ; ; )
    cout << "Hello World\n";
```

نظرًا لعدم وجود طريقة لإيقاف هذه الحلقة، فسوف تعرض "Hello World\n" إلى الأبد (أو حتى يقاطع شيء ما البرنامج).

5.7 Keeping a Running Total

المفهوم: المجموع الجاري هو مجموع أرقام يتراكم مع كل تكرار للحلقة. يُسمى المتغير المستخدم للحفاظ على المجموع الجاري "المُراكم".

تتطلب العديد من مهام البرمجة حساب إجمالي سلسلة من الأرقام. على سبيل المثال، لنفترض أنك تكتب برنامجًا يحسب إجمالي مبيعات شركة لمدة أسبوع. سيقرأ البرنامج مبيعات كل يوم كمدخلات ويحسب إجمالي تلك الأرقام.

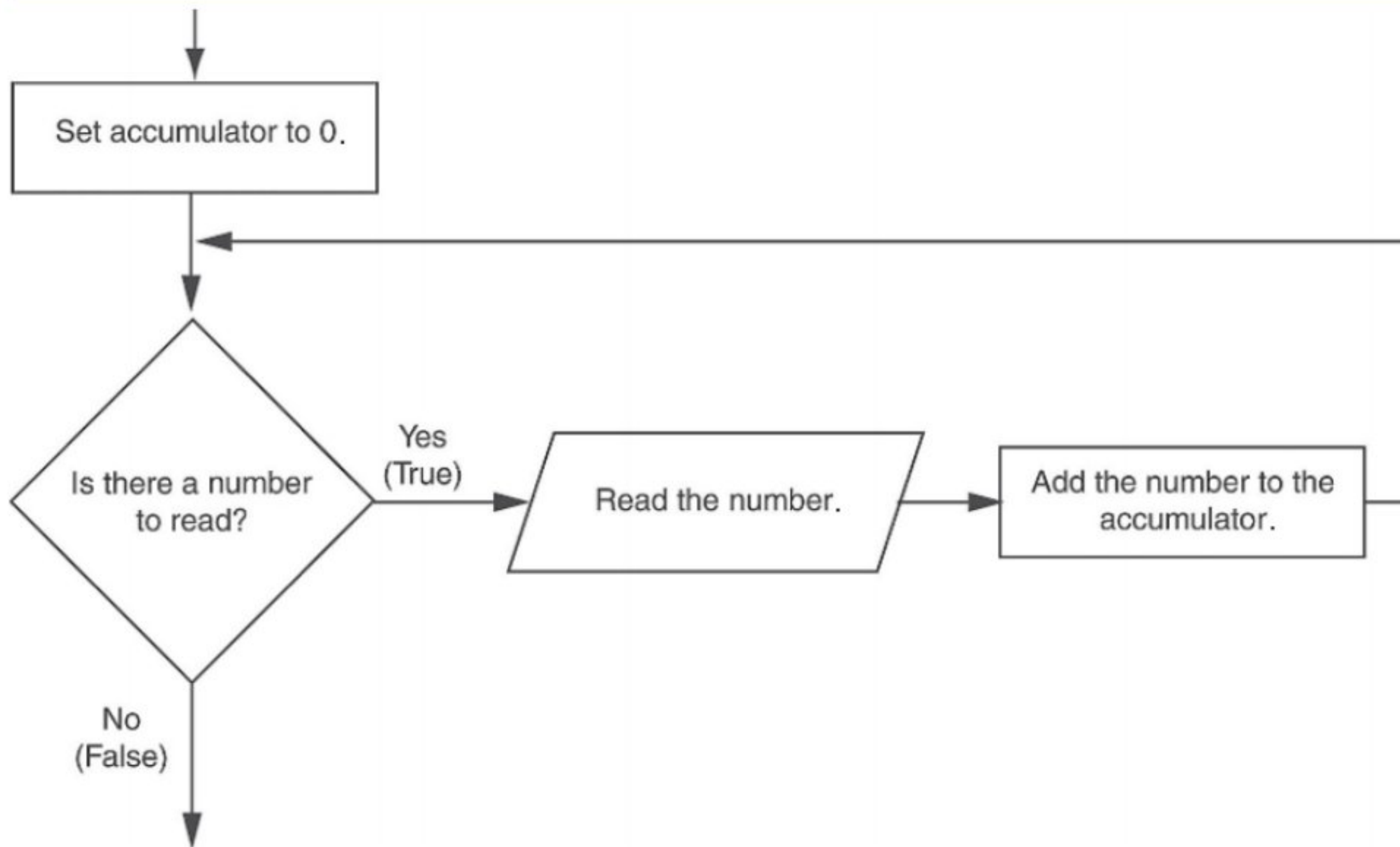
تستخدم البرامج التي تحسب إجمالي سلسلة من الأرقام عادةً عنصرين:

1. حلقة برمجية تقرأ كل رقم في السلسلة.

2. متغير يجمع إجمالي الأرقام أثناء قراءتها.

يُسمى المتغير المستخدم لتجميع إجمالي الأرقام "المُجمع". يُقال غالبًا إن الحلقة البرمجية تحتفظ بإجمالي مستمر لأنها تجمع الإجمالي أثناء قراءتها لكل رقم في السلسلة. يوضح الشكل 5-11 المنطق العام للحلقة البرمجية التي تحسب إجماليًا مستمرًا.

Figure 5-11 Logic for calculating a running total



لنلق نظرة على برنامج يحسب إجمالي المبيعات الجارية. يحسب البرنامج 5-12 إجمالي مبيعات الشركة على مدى فترة زمنية من خلال إدخال أرقام المبيعات اليومية كمدخلات، ثم حساب إجمالي المبيعات الجارية عند جمعها.

Program 5-12

```
1 // This program takes daily sales amounts over a period of time
2 // and calculates their total.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     int days;           // Number of days
10    double total = 0.0; // Accumulator, initialized with 0
11
12    // Get the number of days.
13    cout << "For how many days do you have sales amounts? ";
14
15    cin >> days;
16
17    // Get the sales for each day and accumulate a total.
18    for (int count = 1; count <= days; count++)
19    {
20        double sales;
21        cout << "Enter the sales for day " << count << ": ";
22        cin >> sales;
23        total += sales; // Accumulate the running total.
24    }
25
26    // Display the total sales.
27    cout << fixed << showpoint << setprecision(2);
28    cout << "The total sales are $" << total << endl;
29    return 0;
30 }
```

Program Output with Example Input Shown in Bold

```
For how many days do you have sales amounts? 5  Enter
Enter the sales for day 1: 489.32  Enter
Enter the sales for day 2: 421.65  Enter
Enter the sales for day 3: 497.89  Enter
Enter the sales for day 4: 532.37  Enter
Enter the sales for day 5: 506.92  Enter
The total sales are $2448.15
```

5.8 Sentinels

المفهوم: قيمة "الحارس" (Sentinels) هي قيمة خاصة تُشير إلى نهاية قائمة من القيم.

يتطلب البرنامج 5-12، في القسم السابق، من المستخدم معرفة عدد الأيام التي يرغب في إدخال أرقام المبيعات لها مسبقًا. أحيانًا، تكون قائمة المستخدم طويلة جدًا ولا يعرف عدد العناصر الموجودة فيها. في حالات أخرى، قد يُدخل المستخدم عدة قوائم، ومن غير العملي طلب عدد كل عنصر في كل قائمة.

إحدى التقنيات التي يمكن استخدامها في هذه الحالات هي مطالبة المستخدم بإدخال قيمة "الحارس" في نهاية القائمة. قيمة "الحارس" هي قيمة خاصة لا يمكن الخلط بينها وبين أحد عناصر القائمة، وتُشير إلى عدم وجود قيم أخرى لإدخالها. عند إدخال المستخدم قيمة "الحارس"، تنتهي الحلقة.

يحسب البرنامج 5-13 إجمالي النقاط التي أحرزها فريق كرة قدم في سلسلة من المباريات. إنه يسمح للمستخدم بإدخال سلسلة نقاط اللعبة، ثم 1- للإشارة إلى نهاية القائمة.

Program 5-13

```
1 // This program calculates the total number of points a
2 // soccer team has earned over a series of games. The user
3 // enters a series of point values, then -1 when finished.
4 #include <iostream>
5 using namespace std;
6
7 int main()
8 {
9     int game = 1,    // Game counter
10        points,      // To hold a number of points
11        total = 0;    // Accumulator
12
13     cout << "Enter the number of points your team has earned\n";
14     cout << "so far in the season, then enter -1 when finished.\n\n";
15     cout << "Enter the points for game " << game << ": ";
16     cin >> points;
17
18     while (points != -1)
19     {
20         total += points;
21         game++;
22         cout << "Enter the points for game " << game << ": ";
23         cin >> points;
24     }
25     cout << "\nThe total points are " << total << endl;
26     return 0;
27 }
```

Program Output with Example Input Shown in Bold

Enter the number of points your team has earned so far in the season, then enter -1 when finished.

Enter the points for game 1: **7**
Enter the points for game 2: **9**
Enter the points for game 3: **4**
Enter the points for game 4: **6**
Enter the points for game 5: **8**
Enter the points for game 6: **-1**

The total points are 34

5.9 Focus on Software Engineering: Deciding Which Loop to Use

المفهوم: على الرغم من إمكانية كتابة معظم الخوارزميات التكرارية باستخدام أيٍّ من أنواع الحلقات الثلاثة، إلا أن كل نوع يعمل بشكل أفضل في مواقف مختلفة.

تُعد كل حلقة من حلقات C++ الثلاث مثالية للاستخدام في مواقف مختلفة. إليك ملخصًا موجزًا لاستخدام كل حلقة:

- **حلقة while:** حلقة while هي حلقة شرطية، أي أنها تتكرر طالما وُجد شرط معين. وهي أيضًا حلقة اختبار أولي، لذا فهي مثالية في الحالات التي لا ترغب فيها في تكرار الحلقة إذا كان الشرط خاطئًا من البداية. على سبيل المثال، يُعد التحقق من صحة المدخلات التي تمت قراءتها وقراءة قوائم البيانات التي تنتهي بقيمة مرجعية تطبيقات جيدة لحلقة while.
- **حلقة do-while:** حلقة do-while هي أيضًا حلقة شرطية. على عكس حلقة while، فإن do-while هي حلقة اختبار لاحق. وهي مثالية في الحالات التي ترغب فيها دائمًا في تكرار الحلقة مرة واحدة على الأقل. حلقة do-while خيار جيد لتكرار قائمة.
- **حلقة for:** حلقة for هي حلقة اختبار أولي تحتوي على تعبيرات مدمجة للتهيئة والاختبار والتحديث. تُسهّل هذه التعبيرات استخدام متغير العداد للتحكم في عدد التكرارات التي تُجريها الحلقة. يُمكن لتعبير التهيئة تهيئة متغير العداد إلى قيمة ابتدائية، ويُمكن لتعبير الاختبار اختبار متغير العداد لتحديد ما إذا كان يحمل القيمة القصوى، ويُمكن لتعبير التحديث زيادة متغير العداد. تُعد حلقة for مثالية في الحالات التي يكون فيها عدد التكرارات معروفًا بدقة.

5.10 Nested Loops

المفهوم: تُسمى الحلقة التي تكون داخل حلقة أخرى حلقة متداخلة. الحلقة المتداخلة هي حلقة تظهر داخل حلقة أخرى. الساعة مثال جيد على آلية تعمل كحلقة متداخلة. يدور عقرب الثواني، وعقرب الدقائق، وعقرب الساعات حول وجه الساعة. في كل مرة يزداد فيها عقرب الساعات، يزداد عقرب الدقائق 60 مرة. في كل مرة يزداد فيها عقرب الدقائق، يزداد عقرب الثواني 60 مرة. هذا مقطع برنامج يحتوي على حلقة for تُحاكي جزئياً ساعة رقمية. يعرض الثواني من 0 إلى 59.

```
cout << fixed << right;
cout.fill('0');
for (int seconds = 0; seconds < 60; seconds++)
    cout << setw(2) << seconds << endl;
```



NOTE: The `fill` member function of `cout` changes the fill character, which is a space by default. In the program segment above, the `fill` function causes a zero to be printed in front of all single digit numbers.

يمكننا إضافة متغير الدقائق ووضع الحلقة أعلاه داخل حلقة أخرى تدور لمدة 60 دقيقة.

```
cout << fixed << right;
cout.fill('0');
for (int minutes = 0; minutes < 60; minutes++)
{
    for (int seconds = 0; seconds < 60; seconds++)
    {
        cout << setw(2) << minutes << ":";
        cout << setw(2) << seconds << endl;
    }
}
```

لإكمال محاكاة الساعة، يمكن إضافة متغير وحلقة أخرى لحساب الساعات.

```
cout << fixed << right;
cout.fill('0');
for (int hours = 0; hours < 24; hours++)
{
```

```

for (int minutes = 0; minutes < 60; minutes++)
{
    for (int seconds = 0; seconds < 60; seconds++)
    {
        cout << setw(2) << hours << ":";
        cout << setw(2) << minutes << ":";
        cout << setw(2) << seconds << endl;
    }
}
}

```

The output of the previous program segment follows:

00:00:00

00:00:01

00:00:02

(The program will count through each second of 24 hours.)

.

.

23:59:59

ستتكرر الحلقة الداخلية 60 مرة لكل تكرار للحلقة الوسطى. وستتكرر الحلقة الوسطى 60 مرة لكل تكرار للحلقة الخارجية. عندما تتكرر الحلقة الخارجية 24 مرة، تكون الحلقة الوسطى قد كررت 1440 مرة، وتكون الحلقة الداخلية قد كررت 86400 مرة!

يُظهر مثال محاكاة الساعة بعض النقاط حول الحلقات المتداخلة:

- تُكمل الحلقة الداخلية جميع تكراراتها لكل تكرار للحلقة الخارجية.
- تُكمل الحلقات الداخلية تكراراتها أسرع من الحلقات الخارجية.
- للحصول على العدد الإجمالي لتكرارات الحلقة المتداخلة، اضرب عدد تكرارات جميع الحلقات.

البرنامج 5-14 هو برنامج آخر لحساب متوسط الاختبارات. يطلب من المستخدم عدد الطلاب وعدد درجات الاختبار لكل طالب. تطلب حلقة داخلية متداخلة، في الأسطر من 26 إلى 33، جميع درجات الاختبار لطالب واحد، وتتكرر مرة واحدة لكل درجة اختبار. أما الحلقة الخارجية، في الأسطر من 23 إلى 37، فتتكرر مرة واحدة لكل طالب.

Program 5-14

```
1 // This program averages test scores. It asks the user for the
2 // number of students and the number of test scores per student.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
```

Program 5-14 (continued)

```
7 int main()
8 {
9     int numStudents, // Number of students
10     numTests;       // Number of tests per student
11     double total,    // Accumulator for total scores
12     average;        // Average test score
13
14     // Set up numeric output formatting.
15     cout << fixed << showpoint << setprecision(1);
16
17     // Get the number of students.
18     cout << "This program averages test scores.\n";
19     cout << "For how many students do you have scores? ";
20     cin >> numStudents;
21
22     // Get the number of test scores per student.
23     cout << "How many test scores does each student have? ";
24     cin >> numTests;
25
26     // Determine each student's average score.
27     for (int student = 1; student <= numStudents; student++)
28     {
29         total = 0; // Initialize the accumulator.
30         for (int test = 1; test <= numTests; test++)
31         {
32             double score;
33             cout << "Enter score " << test << " for ";
34             cout << "student " << student << ": ";
35             cin >> score;
36             total += score;
37         }
38         average = total / numTests;
39         cout << "The average score for student " << student;
40         cout << " is " << average << ".\n\n";
41     }
42     return 0;
43 }
```

Program Output with Example Input Shown in Bold

```
This program averages test scores.
For how many students do you have scores? 2  Enter
How many test scores does each student have? 3  Enter
Enter score 1 for student 1: 84  Enter
Enter score 2 for student 1: 79  Enter
Enter score 3 for student 1: 97  Enter
The average score for student 1 is 86.7.

Enter score 1 for student 2: 92  Enter
Enter score 2 for student 2: 88  Enter
Enter score 3 for student 2: 94  Enter
The average score for student 2 is 91.3.
```

5.11 Using Files for Data Storage

المفهوم: عندما يحتاج برنامج إلى حفظ بيانات لاستخدامها لاحقًا، فإنه يكتبها في ملف. يمكن بعد ذلك قراءة البيانات من الملف لاحقًا.

تتطلب البرامج التي كتبتها حتى الآن من المستخدم إعادة إدخال البيانات في كل مرة يتم فيها تشغيل البرنامج، لأن البيانات المحفوظة في المتغيرات وخصائص التحكم تُخزن في ذاكرة الوصول العشوائي (RAM) وتختفي بمجرد توقف البرنامج عن العمل. لكي يحتفظ البرنامج بالبيانات بين مرات تشغيله، يجب أن يكون لديه طريقة لحفظها. تُحفظ البيانات في ملف، والذي عادةً ما يُخزن على قرص الكمبيوتر. بمجرد حفظ البيانات في ملف، ستبقى هناك بعد توقف البرنامج عن العمل. يمكن بعد ذلك استرجاع البيانات المخزنة في ملف واستخدامها لاحقًا.

تخزن معظم البرامج التجارية التي تستخدمها يوميًا البيانات في ملفات. فيما يلي بعض الأمثلة:

- **معالجات النصوص:** تُستخدم برامج معالجة النصوص لكتابة الرسائل والذكرات والتقارير وغيرها من المستندات. تُحفظ المستندات بعد ذلك في ملفات ليتسنى تحريرها وطباعتها لاحقًا.

- **برامج تحرير الصور:** تُستخدم برامج تحرير الصور لرسم الرسومات وتحرير الصور، مثل تلك التي تُلتقط بالكاميرا الرقمية. تُحفظ الصور التي تُنشئها أو تُحررها باستخدام محرر الصور في ملفات.

- **جداول البيانات:** تُستخدم برامج جداول البيانات للتعامل مع البيانات الرقمية. يُمكن إدراج الأرقام والصيغ الرياضية في صفوف وأعمدة جدول البيانات. ثم يُمكن حفظ جدول البيانات في ملف لاستخدامه لاحقًا.

- **الألعاب:** تُخزن العديد من ألعاب الكمبيوتر البيانات في ملفات. على سبيل المثال، تُخزن بعض الألعاب قائمة بأسماء اللاعبين مع نتائجهم في ملف. عادةً ما تعرض هذه الألعاب أسماء اللاعبين مرتبةً حسب نتائجهم، من الأعلى إلى الأدنى. كما تُتيح لك بعض الألعاب حفظ حالة اللعبة الحالية في ملف، ما يُتيح لك إنهاء اللعبة واستئنافها لاحقًا دون الحاجة إلى البدء من البداية.

- **متصفحات الويب:** عند زيارة صفحة ويب، يُخزن المتصفح ملفًا صغيرًا يُعرف باسم ملف تعريف الارتباط على جهاز الكمبيوتر (Cookies). عادةً ما تحتوي ملفات تعريف الارتباط على معلومات حول جلسة التصفح، مثل محتويات عربة التسوق.

تعتمد البرامج المستخدمة في العمليات التجارية اليومية بشكل كبير على الملفات. تُخزن برامج الرواتب بيانات الموظفين في ملفات، وتُخزن برامج الجرد بيانات حول منتجات الشركة في ملفات، وتُخزن أنظمة المحاسبة بيانات حول العمليات المالية للشركة في ملفات، وهكذا.

يُشير المبرمجون عادةً إلى عملية حفظ البيانات في ملف باسم "كتابة البيانات إلى الملف". عند كتابة جزء من البيانات في ملف، تُنسخ من مُتغيّر في ذاكرة الوصول العشوائي (RAM) إلى الملف.

يوضح الشكل 5-12 ذلك. ملف الإخراج هو ملف تُكتب فيه البيانات. ويُسمى ملف إخراج لأن البرنامج يُخزن المخرجات فيه.

تُعرف عملية استرداد البيانات من ملف باسم "قراءة البيانات منه". عند قراءة أي جزء من البيانات من ملف، تُنسخ من الملف إلى متغير في ذاكرة الوصول العشوائي (RAM). يوضح الشكل 5-13 هذه العملية. ملف الإدخال هو الملف الذي تُقرأ منه البيانات. ويُسمى ملف إدخال لأن البرنامج يتلقى البيانات منه.

Figure 5-12 Writing data to a file

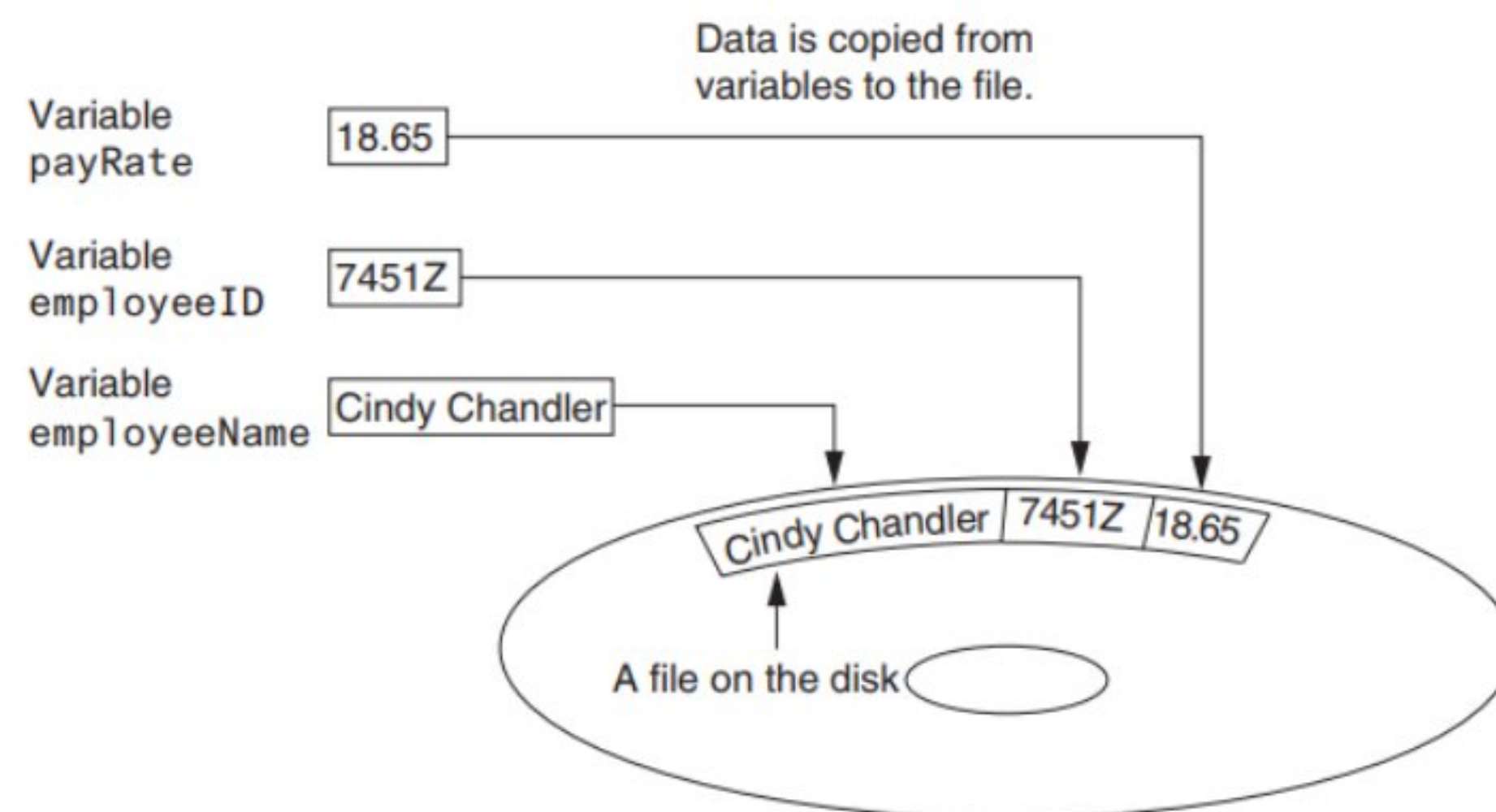
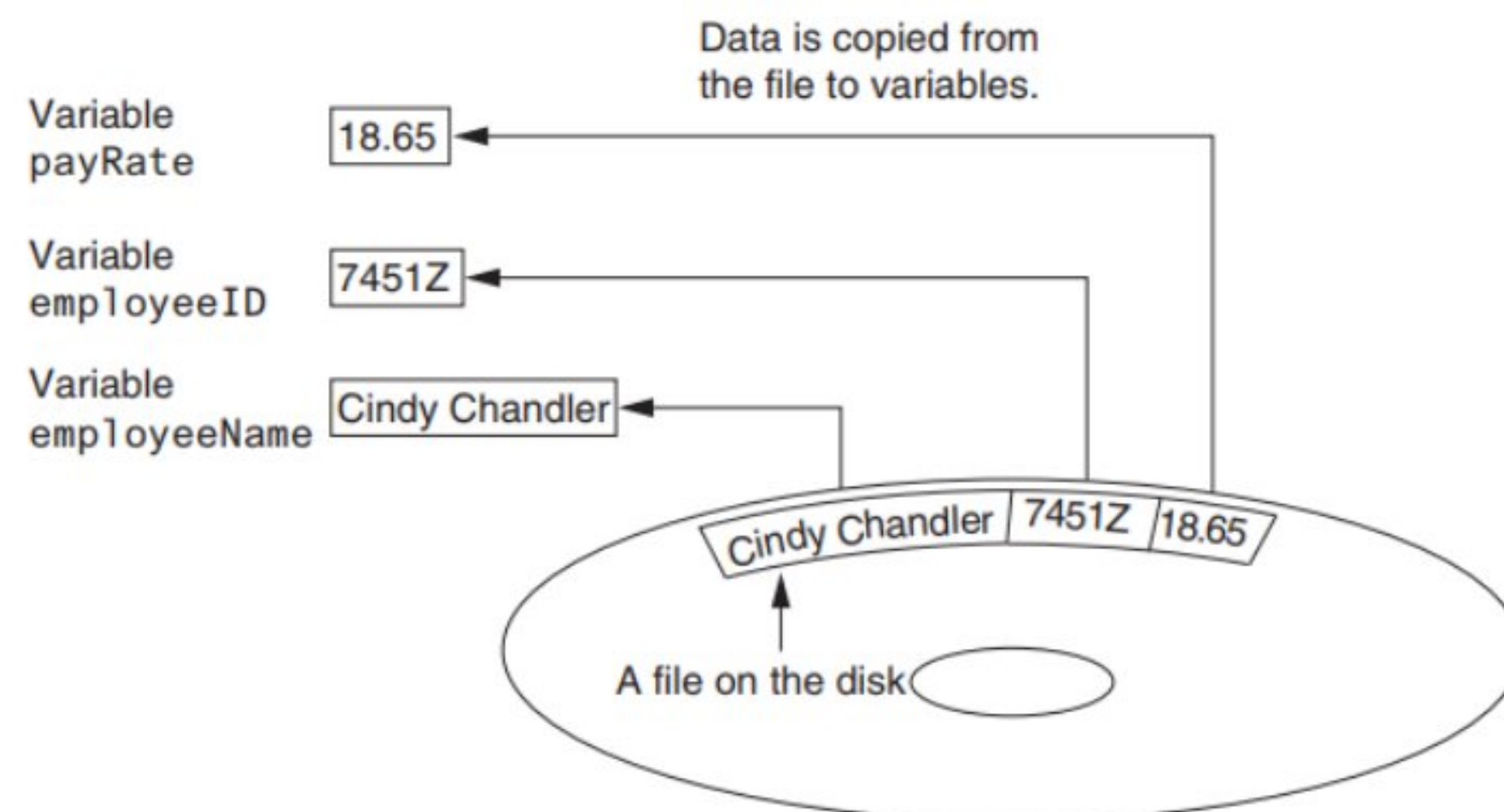


Figure 5-13 Reading data from a file



يناقش هذا القسم طرق إنشاء برامج تكتب البيانات إلى الملفات وتقرأها منها. عند استخدام برنامج ملف، يجب اتباع ثلاث خطوات:

1. **فتح الملف - يُنشئ فتح الملف اتصالاً بينه وبين البرنامج.**
عادةً ما يُنشئ فتح ملف الإخراج الملف على القرص، ويسمح للبرنامج بكتابة البيانات إليه. أما فتح ملف الإدخال، فيسمح للبرنامج بقراءة البيانات منه.
2. **معالجة الملف - تُكتب البيانات إما إلى الملف (إذا كان ملف إخراج) أو تُقرأ منه (إذا كان ملف إدخال).**
3. **إغلاق الملف - بعد انتهاء البرنامج من استخدامه، يجب إغلاقه.**
يؤدي إغلاق الملف إلى فصل اتصاله بالبرنامج.

Types of Files

بشكل عام، هناك نوعان من الملفات: نصية (Text) وثنائية (Binary).

يحتوي الملف النصي على بيانات مُرمّزة كنص، باستخدام نظام مثل ASCII أو Unicode. حتى لو احتوى الملف على أرقام، تُخزّن هذه الأرقام في الملف كسلسلة من الأحرف. ونتيجةً لذلك، يُمكن فتح الملف وعرضه في مُحرّر نصوص مثل Notepad.

يحتوي الملف الثنائي على بيانات لم تُحوّل إلى نص. وبالتالي، لا يُمكنك عرض محتويات الملف الثنائي باستخدام مُحرّر نصوص. في هذه المحاضرة، سنتعامل فقط مع الملفات النصية.

File Access Methods

هناك طريقتان عامتان للوصول إلى البيانات المخزنة في ملف: الوصول التسلسلي والوصول المباشر. **عند العمل مع ملف وصول تسلسلي**، يُمكنك الوصول إلى البيانات من بداية الملف إلى نهايته. إذا كنت ترغب في قراءة جزء من البيانات المخزنة في نهاية الملف، فيجب عليك قراءة جميع البيانات التي تسبقه - لا يُمكنك الانتقال مباشرةً إلى البيانات المطلوبة. هذا مشابه لطريقة عمل مشغلات أشرطة الكاسيت القديمة. إذا كنت ترغب في الاستماع إلى آخر أغنية على شريط كاسيت، فيجب عليك إما التقديم السريع لجميع الأغاني التي تسبقها أو الاستماع إليها. لا توجد طريقة للانتقال مباشرةً إلى أغنية محددة.

عند العمل مع ملف وصول عشوائي (يُعرف أيضاً باسم ملف الوصول المباشر)، يُمكنك الانتقال مباشرةً إلى أي جزء من البيانات في الملف دون قراءة البيانات التي تسبقه. هذا مشابه لطريقة عمل مشغل الأقراص المضغوطة أو مشغل MP3. يُمكنك الانتقال مباشرةً إلى أي أغنية ترغب بسماعها.

تركز هذه المحاضرة على ملفات الوصول التسلسلي. يسهل التعامل مع هذه الملفات، ويُمكنك استخدامها لفهم العمليات الأساسية للملفات.

Filenames and File Stream Objects

يتم تحديد الملفات الموجودة على القرص باسم الملف. على سبيل المثال، عند إنشاء مستند باستخدام معالج نصوص ثم حفظه في ملف، يجب عليك تحديد اسم الملف. عند استخدام أداة مساعدة مثل مستكشف Windows لفحص محتويات القرص، ستري قائمة بأسماء الملفات. يوضح الشكل 5-14 كيفية تمثيل ثلاثة ملفات باسم cat.jpg و notes.txt و resume.doc في مستكشف Windows.

Figure 5-14 Three files



لكي يعمل برنامج مع ملف على قرص الحاسوب، يجب عليه إنشاء كائن تدفق ملفات (file stream object) في الذاكرة. كائن تدفق الملفات هو كائن مرتبط بملف معين، ويُمكن البرنامج من العمل معه. يُسمى كائن "تدفق" لأنه يُمكن اعتبار الملف تدفقًا للبيانات.

تعمل كائنات تدفق الملفات بشكل مشابه جدًا لكائني cin و cout. يُمكن إرسال تدفق بيانات إلى cout، مما يؤدي إلى عرض القيم على الشاشة. يُمكن قراءة تدفق البيانات من لوحة المفاتيح بواسطة cin، وتخزينها في متغيرات. وبالمثل، يُمكن إرسال تدفقات البيانات إلى كائن تدفق الملفات، الذي يكتب البيانات إلى ملف. عند قراءة البيانات من ملف، تتدفق البيانات من كائن تدفق الملفات المرتبط بالملف إلى متغيرات.

Setting Up a Program for File Input/Output

كما يتطلب كلٌّ من cin و cout تضمين ملف iostream في البرنامج، يتطلب الوصول إلى ملف C++ ملف رأس آخر. يحتوي الملف <fstream> على جميع التصريحات اللازمة لعمليات الملف. وهو مُضمّن مع العبارة التالية:

```
#include <fstream>
```

يُعرّف ملف الرأس <fstream> أنواع البيانات التالية: stream، ifstream، و ofstream.

قبل أن يتمكن برنامج C++ من العمل مع ملف، يجب أن يُعرّف كائنًا من أحد هذه الأنواع. سيتم "ربط" الكائن بملف فعلي على قرص الكمبيوتر، وتعتمد العمليات التي يُمكن إجراؤها على الملف على نوع البيانات الثلاثة الذي تختاره لكائن تدفق الملف. يسرد الجدول 5-1 أنواع بيانات تدفق الملف ويصفها.

Table 5-1 File Stream Objects

File Stream Data Type	Description
ofstream	Output file stream. You create an object of this data type when you want to create a file and write data to it.
ifstream	Input file stream. You create an object of this data type when you want to open an existing file and read data from it.
fstream	File stream. Objects of this data type can be used to open files for reading, writing, or both.

ملاحظة: في هذه المحاضرة، نناقش فقط نوعي `ofstream` و `ifstream`.

Creating a File Object and Opening a File

قبل كتابة البيانات إلى ملف أو قراءتها منه، يجب تنفيذ الخطوات التالية:

- إنشاء كائن تدفق ملف.
 - فتح الملف وربطه بكائن تدفق الملف.
- يوضح الكود التالي مثلاً على فتح ملف للإدخال (القراءة).

```
ifstream inputFile;
inputFile.open("Customers.txt");
```

تُعرّف العبارة الأولى كائن `ifstream` باسم `inputFile`. تستدعي العبارة الثانية دالة العضو `open` للكائن، مع تمرير السلسلة النصية `"Customers.txt"` كمعامل. في هذه العبارة، تفتح دالة العضو `open` ملف `Customers.txt` وتربطه بكائن `inputFile`. بعد تنفيذ هذه التعليمات البرمجية، ستتمكن من استخدام كائن `inputFile` لقراءة البيانات من ملف `Customers.txt`.

يُظهر الكود التالي مثلاً لفتح ملف للإخراج (الكتابة).

```
ofstream outputFile;
outputFile.open("Employees.txt");
```

تُعرّف العبارة الأولى كائنًا من نوع `ofstream` يُسمى `outputFile`. أما العبارة الثانية فتستدعي دالة العضو `(open)` للكائن، مُمرّرةً السلسلة النصية `"Employees.txt"` كمعامل. في هذه العبارة، تُنشئ دالة العضو `(open)` ملف `Employees.txt` وتربطه بكائن `outputFile`. بعد تنفيذ هذه التعليمات البرمجية، ستتمكن من استخدام كائن `outputFile` لكتابة البيانات إلى ملف `Employees.txt`. من المهم تذكر أنه عند استدعاء دالة العضو `(open)` لكائن `ofstream`، سيتم إنشاء الملف المُحدد. إذا كان الملف المُحدد موجودًا بالفعل، فسيتم حذفه، وسيتم إنشاء ملف جديد بنفس الاسم. في كثير من الأحيان، عند فتح ملف، ستحتاج إلى تحديد مساره واسمه. على سبيل المثال، في نظام ويندوز، تفتح العبارة التالية الملف `C:\data\inventory.txt`:

```
inputFile.open("C:\\data\\inventory.txt")
```

في هذه العبارة، يُفتح الملف C:\data\inventory.txt ويُربط بملف inputFile. من الممكن تعريف كائن تدفق ملف وفتح ملف في جملة واحدة. إليك مثال:

```
ifstream inputFile("Customers.txt");
```

تُعرّف هذه الجملة كائن تدفق ifstream باسم inputFile وتفتح ملف Customer.txt. إليك مثال يُعرّف كائن تدفق ofstream باسم outputFile ويفتح ملف Employees.txt:

```
ofstream outputFile("Employees.txt");
```

Closing a File

عكس فتح الملف هو إغلاقه. على الرغم من أن ملفات البرنامج تُغلق تلقائيًا عند إيقاف تشغيله، إلا أن كتابة تعليمات لإغلاقها يُعدّ ممارسة برمجية جيدة. إليك سببان لإغلاق البرنامج للملفات عند الانتهاء من استخدامها:

1. تُخزن معظم أنظمة التشغيل البيانات مؤقتًا في مخزن مؤقت (file buffer) للملفات قبل كتابتها فيه. مخزن الملفات هو "قسم تخزين" صغير في الذاكرة تُكتب إليه البيانات المرتبطة بالملف أولاً. عند امتلاء المخزن المؤقت، تُكتب جميع البيانات المخزنة فيه إلى الملف.

تُحسن هذه التقنية أداء النظام. يؤدي إغلاق الملف إلى حفظ أي بيانات غير محفوظة قد لا تزال موجودة في المخزن المؤقت في ملفه. هذا يعني أن البيانات ستكون موجودة في الملف إذا احتجت إلى قراءتها لاحقًا في نفس البرنامج.

2. تُحدّد بعض أنظمة التشغيل عدد الملفات التي يُمكن فتحها في وقت واحد. عندما يُغلق البرنامج الملفات التي لم تعد تُستخدم، فلن يُستنزف ذلك موارد نظام التشغيل أكثر من اللازم.

يؤدي استدعاء دالة إغلاق كائن تدفق الملفات إلى إغلاق الملف. إليك مثال:

```
inputFile.close();
```

Writing Data to a File

أنت تعرف بالفعل كيفية استخدام مُعامل إدراج التدفق (>>) مع كائن cout لكتابة البيانات على الشاشة. كما يمكن استخدامه مع كائنات ofstream لكتابة البيانات إلى ملف.

بافتراض أن outputFile كائن من نوع ofstream، توضح العبارة التالية استخدام مُعامل >> لكتابة نص حرفي إلى ملف:

```
outputFile << "I love C++ programming\n";
```

تكتب هذه العبارة النص الحرفي "I love C++ programming\n" إلى الملف المرتبط بـ outputFile. كما ترى، تبدو العبارة كجملة cout، باستثناء أن اسم كائن ofstream يحل محل cout. إليك عبارة تكتب نصًا حرفيًا ومحتويات متغير إلى ملف:

```
outputFile << "Price: " << price << endl;
```

تكتب العبارة أعلاه تدفق البيانات إلى ملف الإخراج تمامًا كما يكتبها الدالة cout على الشاشة: تكتب السلسلة النصية "Price: "، متبوعة بقيمة متغير السعر، متبوعة بحرف سطر جديد.

يوضح البرنامج 5-15 فتح ملف، وكتابة بيانات فيه، وإغلاقه. بعد تنفيذ هذه التعليمات البرمجية، يمكننا فتح ملف demofile.txt باستخدام محرر نصوص والاطلاع على محتوياته. يوضح الشكل 5-15 كيفية ظهور محتويات الملف في برنامج المفكرة.

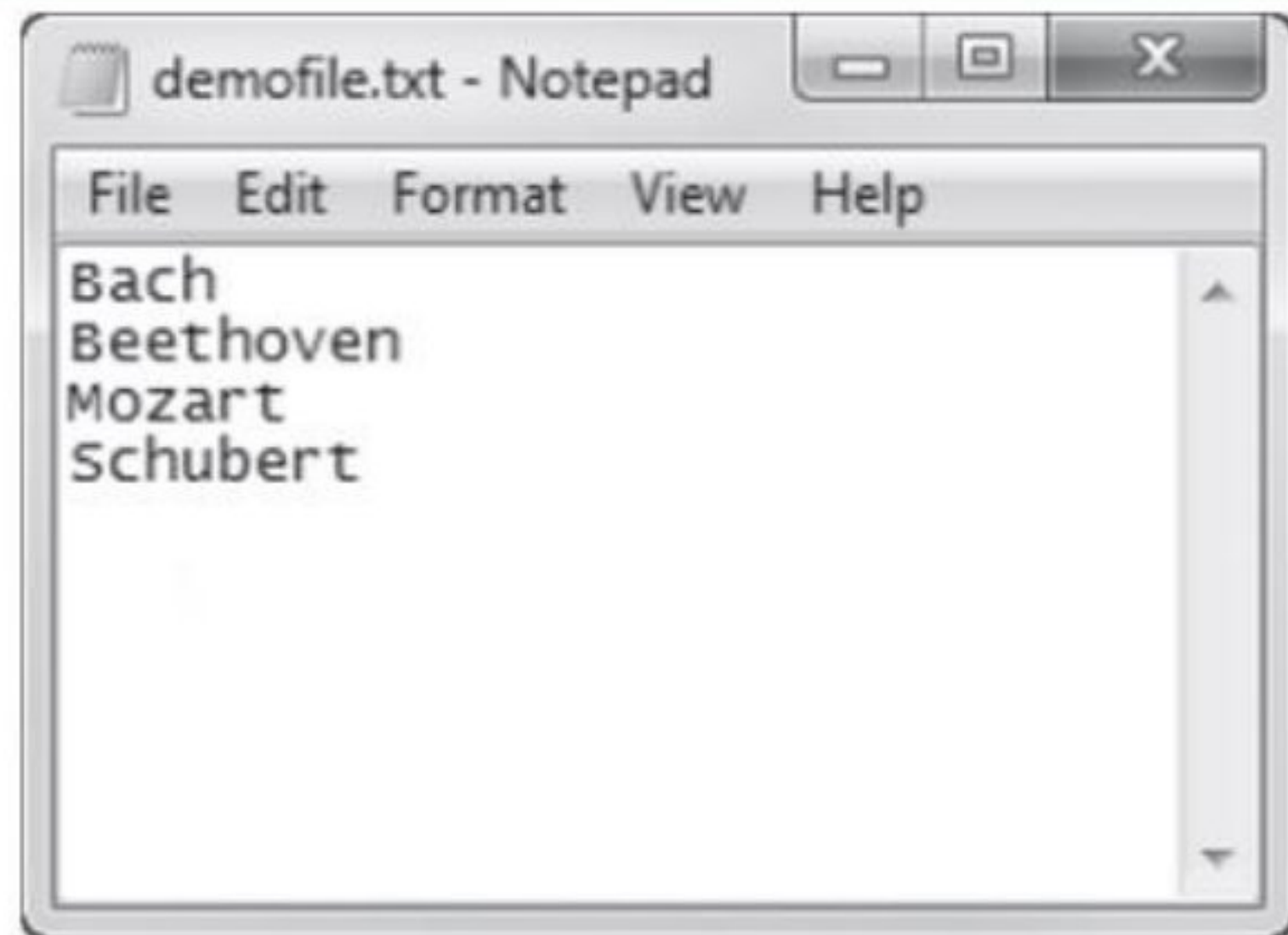
Program 5-15

```
1 // This program writes data to a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ofstream outputFile;
9     outputFile.open("demofile.txt");
10
11     cout << "Now writing data to the file.\n";
12
13     // Write four names to the file.
14     outputFile << "Bach\n";
15     outputFile << "Beethoven\n";
16     outputFile << "Mozart\n";
17     outputFile << "Schubert\n";
18
19     // Close the file
20     outputFile.close();
21     cout << "Done.\n";
22     return 0;
23 }
```

Program Screen Output

```
Now writing data to the file.
Done.
```

Figure 5-15 Contents of demofile.txt



يوضح البرنامج 5-16 ما يحدث إذا كتبنا نفس الأسماء الأربعة بدون تسلسل الهروب \n. يوضح الشكل 5-16 محتويات الملف الذي يُنشئه البرنامج 5-16. كما ترى، تظهر جميع الأسماء على نفس السطر في الملف.

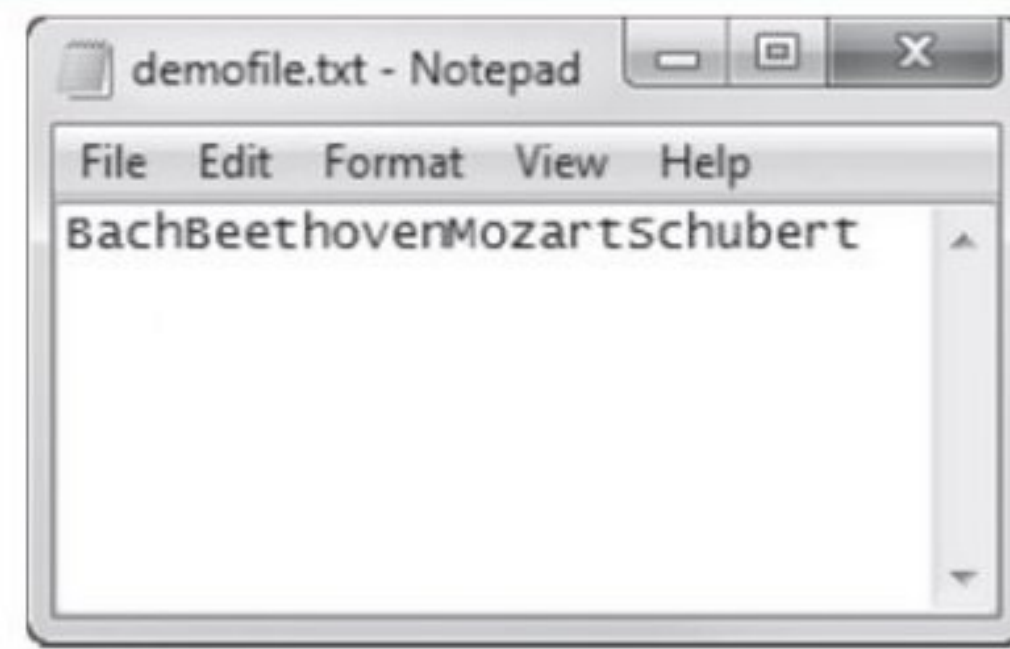
Program 5-16

```
1 // This program writes data to a single line in a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ofstream outputFile;
9     outputFile.open("demofile.txt");
10
11     cout << "Now writing data to the file.\n";
12
13     // Write four names to the file.
14     outputFile << "Bach";
15     outputFile << "Beethoven";
16     outputFile << "Mozart";
17     outputFile << "Schubert";
18
19     // Close the file
20     outputFile.close();
21     cout << "Done.\n";
22     return 0;
23 }
```

Program Screen Output

```
Now writing data to the file.
Done.
```

Figure 5-16 Output written as one line



يُظهر البرنامج 5-17 مثلاً آخر. يقرأ هذا البرنامج ثلاثة أرقام من لوحة المفاتيح كمدخلات، ثم يحفظها في ملف باسم Numbers.txt.

Program 5-17

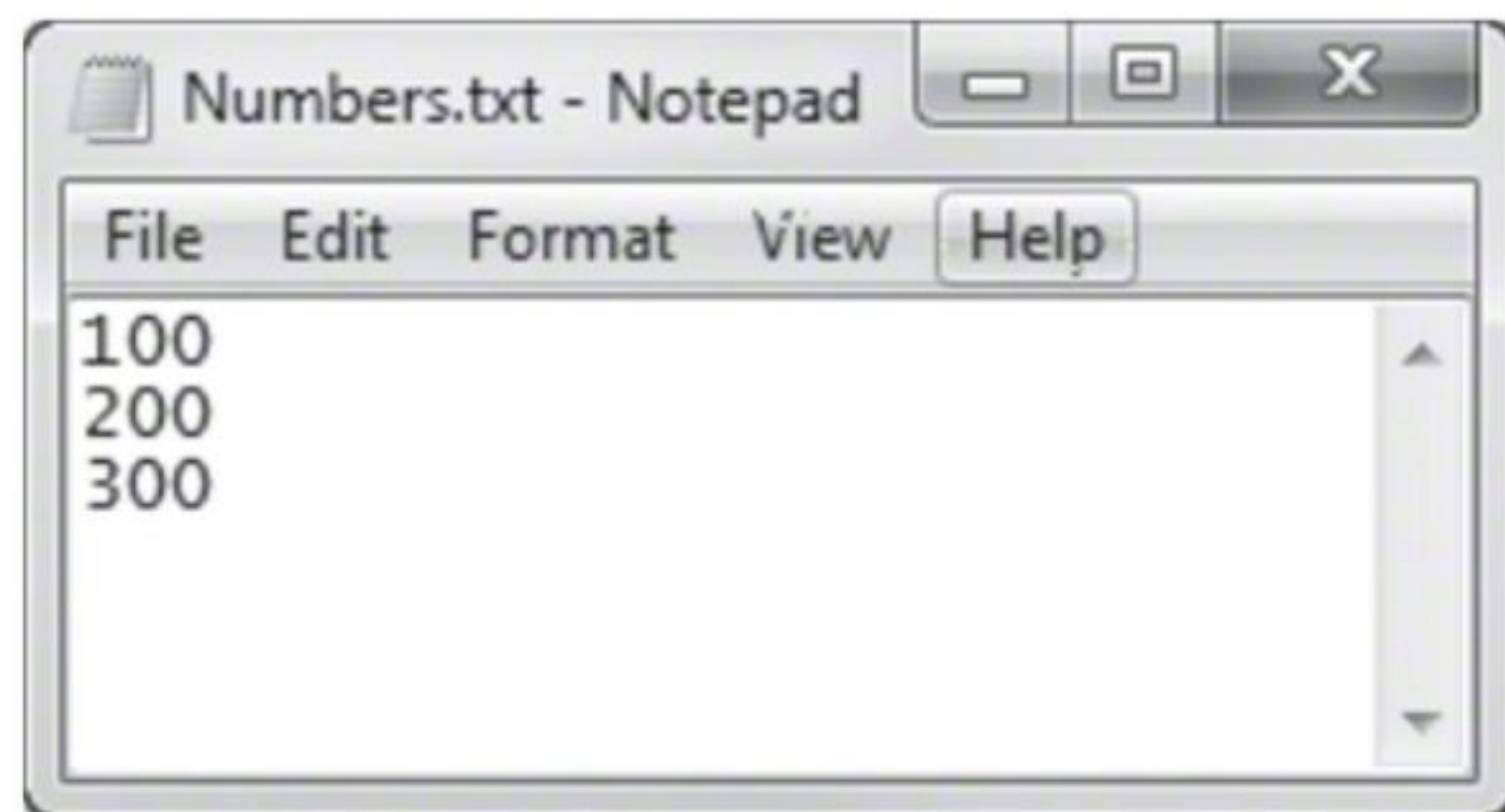
```
1 // This program writes user input to a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ofstream outputFile;
9     int number1, number2, number3;
10
11     // Open an output file.
12     outputFile.open("Numbers.txt");
13
14     // Get three numbers from the user.
15     cout << "Enter a number: ";
16     cin >> number1;
17     cout << "Enter another number: ";
18     cin >> number2;
19     cout << "One more time. Enter a number: ";
20     cin >> number3;
21
22     // Write the numbers to the file.
23     outputFile << number1 << endl;
24     outputFile << number2 << endl;
25     outputFile << number3 << endl;
26     cout << "The numbers were saved to a file.\n";
27
28     // Close the file
29     outputFile.close();
30     cout << "Done.\n";
31     return 0;
32 }
```

Program Screen Output with Example Input Shown in Bold

```
Enter a number: 100   
Enter another number: 200   
One more time. Enter a number: 300   
The numbers were saved to a file.  
Done.
```

في البرنامج 5-17، تُكتب الأسطر من 23 إلى 25 محتويات المتغيرات number1 و number2 و number3 في الملف. لاحظ أن مُعالج endl يُرسل إلى كائن outputFile مباشرةً بعد كل عنصر. يؤدي إرسال مُعالج endl إلى كتابة سطر جديد في الملف. يُظهر الشكل 5-17 محتويات الملف المعروضة في المفكرة، باستخدام قيم الإدخال 100 و 200 و 300 كمثال. كما ترى، يظهر كل عنصر في سطر منفصل في الملف بفضل مُعالجات endl.

Figure 5-17 Contents of Numbers.txt



يُظهر البرنامج 5-18 مثلاً يقرأ سلاسل نصية كمدخلات من لوحة المفاتيح، ثم يكتبها في ملف. يطلب البرنامج من المستخدم إدخال الأسماء الأولى لثلاثة أصدقاء، ثم يكتبها في ملف Friends.txt. يوضح الشكل 5-18 مثلاً لملف Friends.txt المفتوح في برنامج المفكرة.

Program 5-18

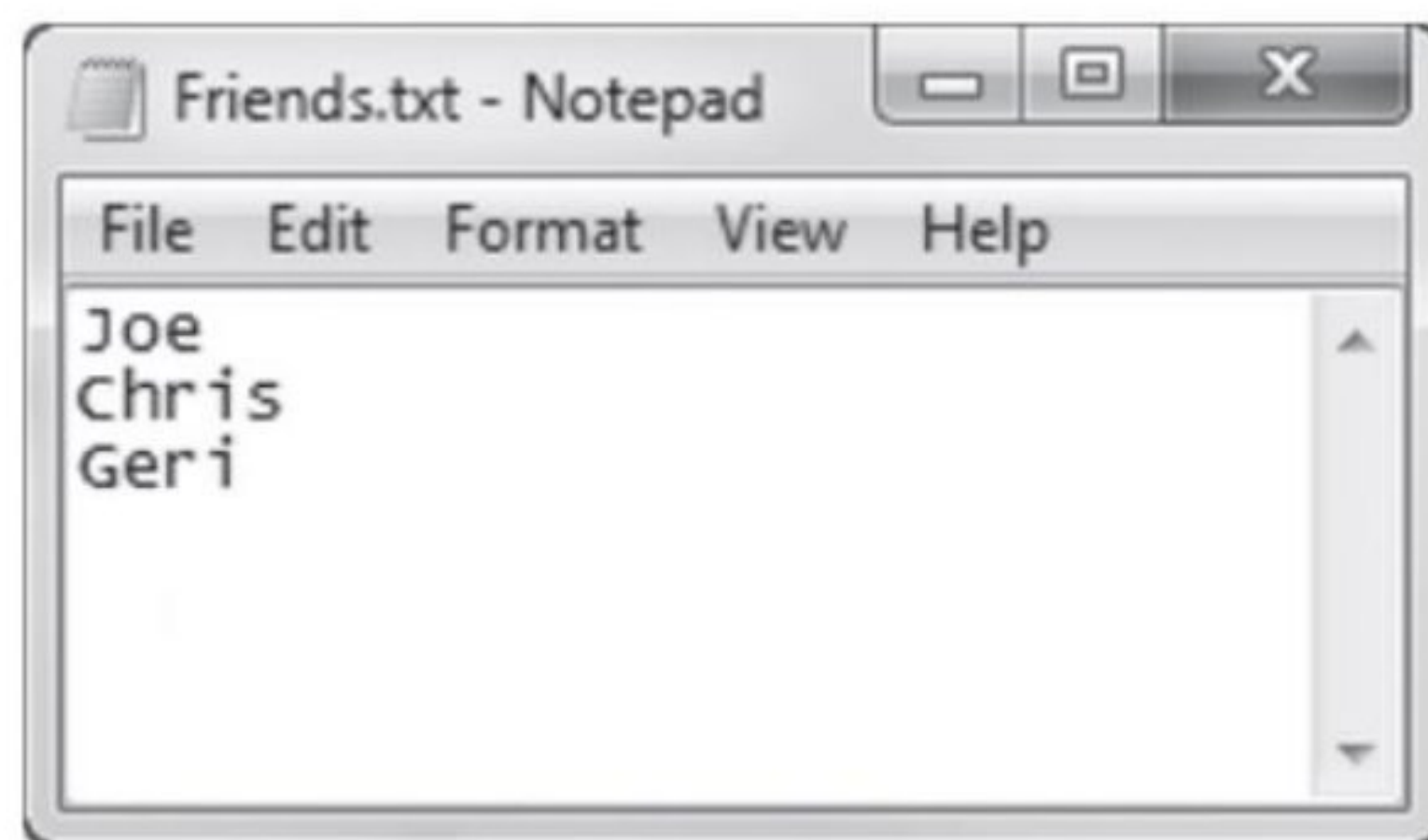
```
1 // This program writes user input to a file.  
2 #include <iostream>  
3 #include <fstream>  
4 #include <string>  
5 using namespace std;  
6  
7 int main()  
8 {  
9     ofstream outputFile;  
10    string name1, name2, name3;  
11  
12    // Open an output file.  
13    outputFile.open("Friends.txt");  
14
```

Program 5-18 (continued)

```
15 // Get the names of three friends.
16 cout << "Enter the names of three friends.\n";
17 cout << "Friend #1: ";
18 cin >> name1;
19 cout << "Friend #2: ";
20 cin >> name2;
21 cout << "Friend #3: ";
22 cin >> name3;
23
24 // Write the names to the file.
25 outputFile << name1 << endl;
26 outputFile << name2 << endl;
27 outputFile << name3 << endl;
28 cout << "The names were saved to a file.\n";
29
30 // Close the file
31 outputFile.close();
32 return 0;
33 }
```

Program Screen Output with Example Input Shown in Bold

```
Enter the names of three friends.
Friend #1: Joe 
Friend #2: Chris 
Friend #3: Geri 
The names were saved to a file.
```

Figure 5-18 Contents of Friends.txt**Reading Data from a File**

لا يقرأ عامل التشغيل >> مُدخلات المستخدم من كائن cin فحسب، بل يقرأ أيضًا بيانات من ملف.

بافتراض أن ملف الإدخال هو كائن تدفق ifstream، تُظهر العبارة التالية عامل التشغيل >> وهو يقرأ البيانات من الملف إلى اسم المتغير:

inputFile >> name;

لنلق نظرة على مثال. لنفترض وجود ملف Friends.txt، وأنه يحتوي على الأسماء الموضحة في الشكل 5-18. يفتح البرنامج 5-19 الملف، ويقرأ الأسماء ويعرضها على الشاشة، ثم يُغلقه.

Program 5-19

```
1 // This program reads data from a file.
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5 using namespace std;
6
7 int main()
8 {
9     ifstream inputFile;
10    string name;
11
12    inputFile.open("Friends.txt");
13    cout << "Reading data from the file.\n";
14
15    inputFile >> name;        // Read name 1 from the file
16    cout << name << endl;     // Display name 1
17
18    inputFile >> name;        // Read name 2 from the file
19    cout << name << endl;     // Display name 2
20
21    inputFile >> name;        // Read name 3 from the file
22    cout << name << endl;     // Display name 3
23
24    inputFile.close();        // Close the file
25    return 0;
26 }
```

Program Output

```
Reading data from the file.
Joe
Chris
Geri
```

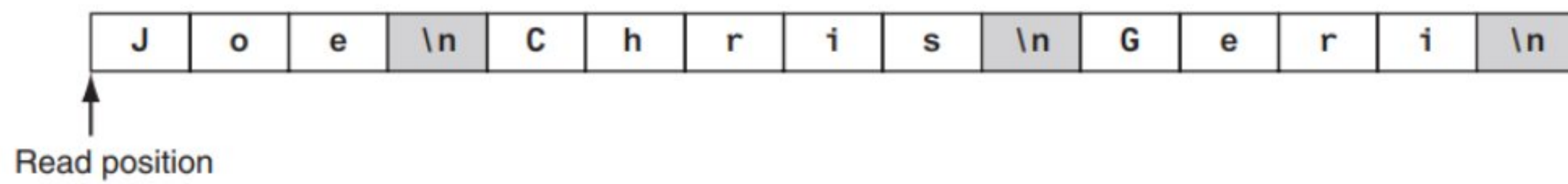
The Read Position

عند فتح ملف للإدخال، يحتفظ كائن تدفق الملف داخليًا بقيمة خاصة تُعرف بموضع القراءة (Read Position). يُحدد موضع قراءة الملف موقع البايت التالي الذي سيُقرأ منه. عند فتح ملف إدخال، يُضبط موضع القراءة مبدئيًا على البايت الأول في الملف. لذا، تستخرج عملية القراءة الأولى البيانات بدءًا من البايت الأول.

عند قراءة البيانات من الملف، يتحرك موضع القراءة للأمام، نحو نهاية الملف.

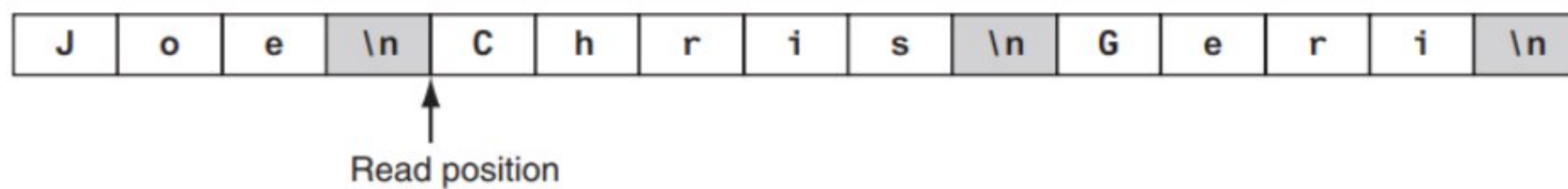
لنر كيف يعمل هذا مع المثال الموضح في البرنامج 5-19. عند فتح ملف Friends.txt باستخدام العبارة في السطر 12، سيتم وضع موضع قراءة الملف كما هو موضح في الشكل 5-19.

Figure 5-19 Initial read position



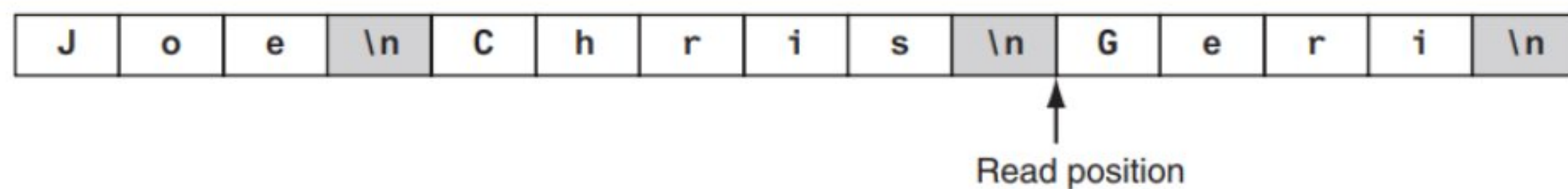
تذكر أنه عند استخراج (>>) بيانات من ملف، فإنه يتوقع قراءة أجزاء من البيانات مفصولة بمسافات بيضاء (مسافات (spaces)، أو علامات تبويب (tabs)، أو أسطر جديدة (newlines)). عند تنفيذ العبارة في السطر 15، يقرأ عامل >> البيانات من موضع القراءة الحالي للملف، وصولاً إلى الحرف \n. تُخصص البيانات المقروءة من الملف لكائن الاسم. يُقرأ الحرف \n أيضاً من الملف، ولكنه لا يُضمّن كجزء من البيانات. لذا، سيحتفظ كائن الاسم بالقيمة "Joe" بعد تنفيذ هذه العبارة. عندئذٍ، سيكون موضع قراءة الملف في الموقع الموضح في الشكل 5-20.

Figure 5-20 The next read position



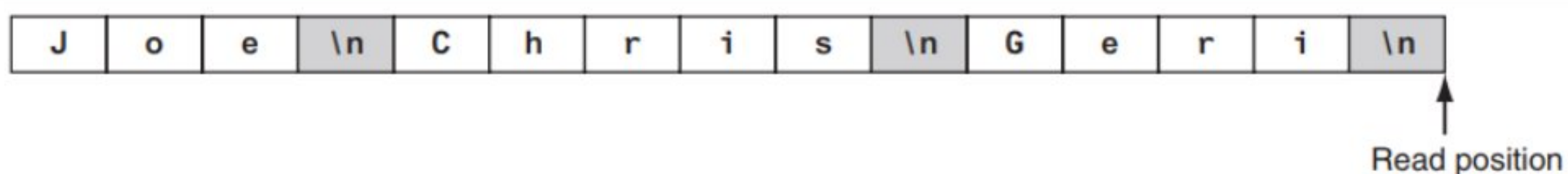
عند تنفيذ العبارة في السطر 18، تقرأ العنصر التالي من الملف، وهو "Chris"، وتُعيّن هذه القيمة لكائن الاسم. بعد تنفيذ هذه العبارة، سيتم نقل موضع قراءة الملف إلى العنصر التالي، كما هو موضح في الشكل 5-21.

Figure 5-21 The next read position



عند تنفيذ العبارة في السطر 21، فإنها تقرأ العنصر التالي من الملف، وهو "Geri"، وتُعيّن هذه القيمة لكائن الاسم. بعد تنفيذ هذه العبارة، سيتم نقل موضع قراءة الملف إلى نهاية الملف، كما هو موضح في الشكل 5-22.

Figure 5-22 The read position at the end of the file

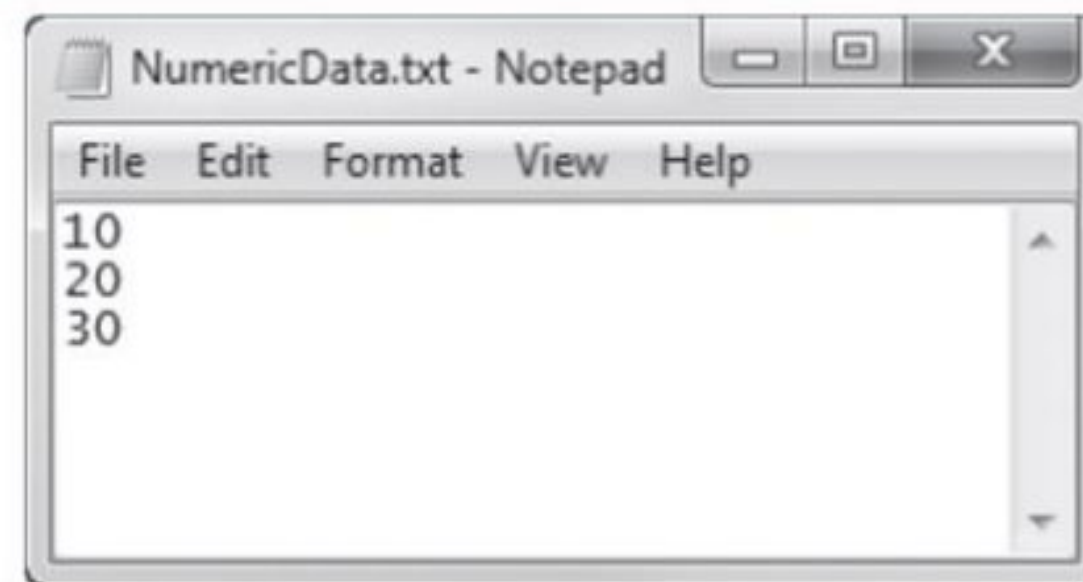


Reading Numeric Data from a Text File

تذكر أنه عند تخزين البيانات في ملف نصي، تُرمز كنص، باستخدام نظام ترميز مثل ASCII أو Unicode. حتى لو احتوى الملف على أرقام، تُخزن هذه الأرقام في الملف كسلسلة من الأحرف. على سبيل المثال، لنفترض أن ملفاً نصياً يحتوي على بيانات رقمية، مثل تلك الموضحة في الشكل 5-17. تُخزن الأرقام المعروضة في الشكل في الملف كسلاسل نصية "10" و"20" و"30". لحسن الحظ،

يمكنك استخدام عامل التشغيل >> لقراءة بيانات كهذه من ملف نصي إلى متغير رقمي، وسيُحوّل عامل التشغيل >> البيانات تلقائيًا إلى نوع بيانات رقمي. يعرض البرنامج 20-5 مثالًا. يفتح البرنامج الملف الموضح في الشكل 5-23، ويقرأ الأرقام الثلاثة من الملف إلى متغيرات من نوع int، ويحسب مجموعها.

Figure 5-23 Contents of NumericData.txt



Program 5-20

```
1 // This program reads numbers from a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ifstream inFile;
9     int value1, value2, value3, sum;
10
11     // Open the file.
12     inFile.open("NumericData.txt");
13
14     // Read the three numbers from the file.
15     inFile >> value1;
16     inFile >> value2;
17     inFile >> value3;
18
19     // Close the file.
20     inFile.close();
21
22     // Calculate the sum of the numbers.
23     sum = value1 + value2 + value3;
24
25     // Display the three numbers.
26     cout << "Here are the numbers:\n"
27           << value1 << " " << value2
28           << " " << value3 << endl;
29
30     // Display the sum of the numbers.
31     cout << "Their sum is: " << sum << endl;
32     return 0;
33 }
```

Program Output

```
Here are the numbers:
10 20 30
Their sum is: 60
```

Using Loops to Process Files

على الرغم من أن بعض البرامج تستخدم الملفات لتخزين كميات صغيرة فقط من البيانات، إلا أنها تُستخدم عادةً للاحتفاظ بمجموعات كبيرة من البيانات. عندما يستخدم برنامج ملفًا لكتابة أو قراءة كمية كبيرة من البيانات، عادةً ما تتضمن حلقة تكرار. على سبيل المثال، انظر إلى الكود في البرنامج 5-21. يحصل هذا البرنامج على مبالغ مبيعات لسلسلة من الأيام من المستخدم، ويكتب هذه المبالغ في ملف باسم Sales.txt. يحدد المستخدم عدد أيام بيانات المبيعات التي يحتاج إلى إدخالها. في التشغيل التجريبي للبرنامج، يُدخل المستخدم مبالغ مبيعات لخمس أيام. يوضح الشكل 5-24 محتويات ملف Sales.txt الذي يحتوي على البيانات التي أدخلها المستخدم في التشغيل التجريبي.

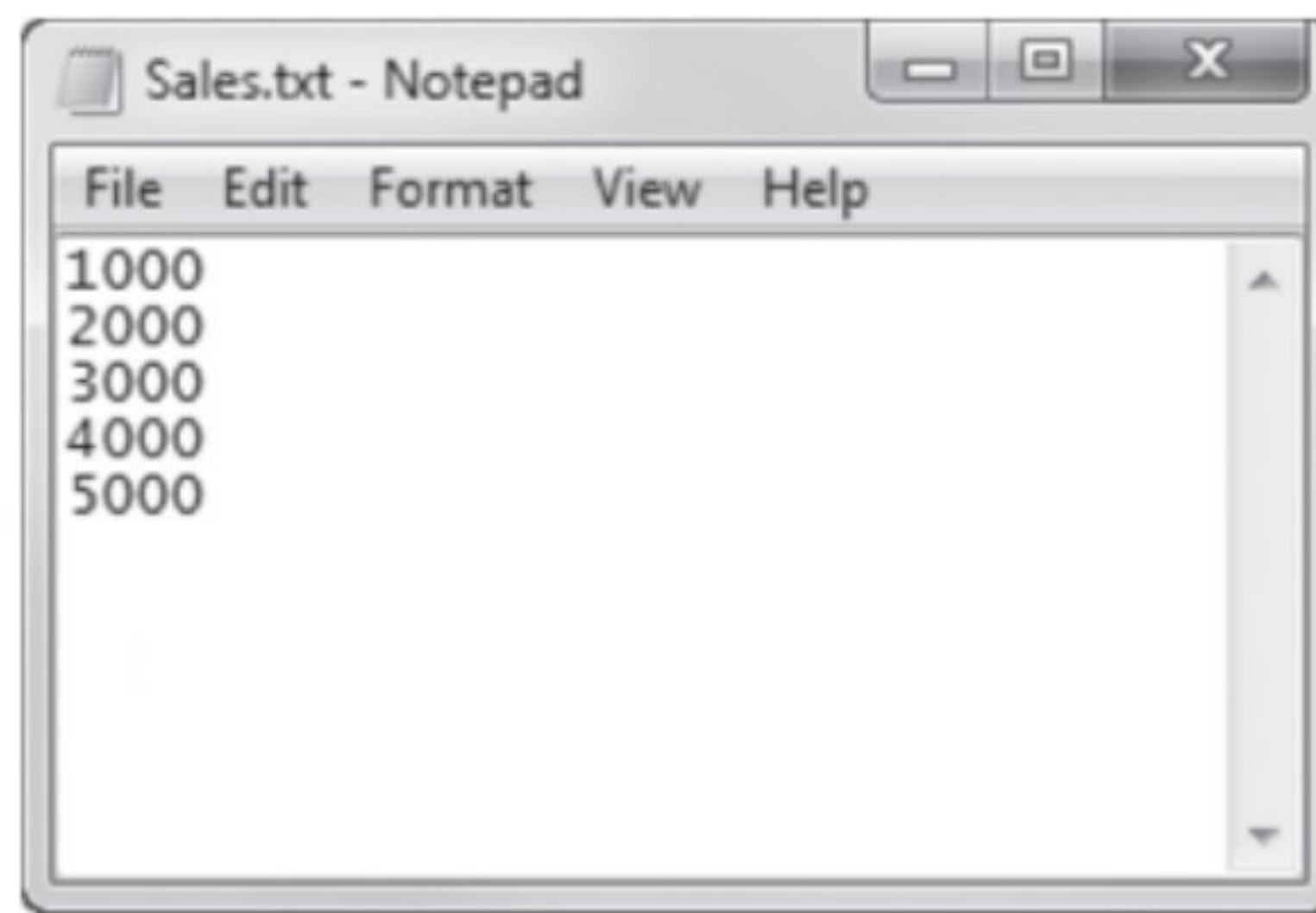
Program 5-21

```
1 // This program reads data from a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ofstream outputFile; // File stream object
9     int numberOfDays;     // Number of days of sales
10    double sales;         // Sales amount for a day
11
12    // Get the number of days.
13    cout << "For how many days do you have sales? ";
14    cin >> numberOfDays;
15
16    // Open a file named Sales.txt.
17    outputFile.open("Sales.txt");
18
19    // Get the sales for each day and write it
20    // to the file.
21    for (int count = 1; count <= numberOfDays; count++)
22    {
23        // Get the sales for a day.
24        cout << "Enter the sales for day "
25              << count << ": ";
26        cin >> sales;
27
28        // Write the sales to the file.
29        outputFile << sales << endl;
30    }
31
32    // Close the file.
33    outputFile.close();
34    cout << "Data written to Sales.txt\n";
35    return 0;
36 }
```

Program Output (with Input Shown in Bold)

```
For how many days do you have sales? 5   
Enter the sales for day 1: 1000.00   
Enter the sales for day 2: 2000.00   
Enter the sales for day 3: 3000.00   
Enter the sales for day 4: 4000.00   
Enter the sales for day 5: 5000.00   
Data written to sales.txt.
```

Figure 5-24 Contents of Sales.txt



Detecting the End of the File

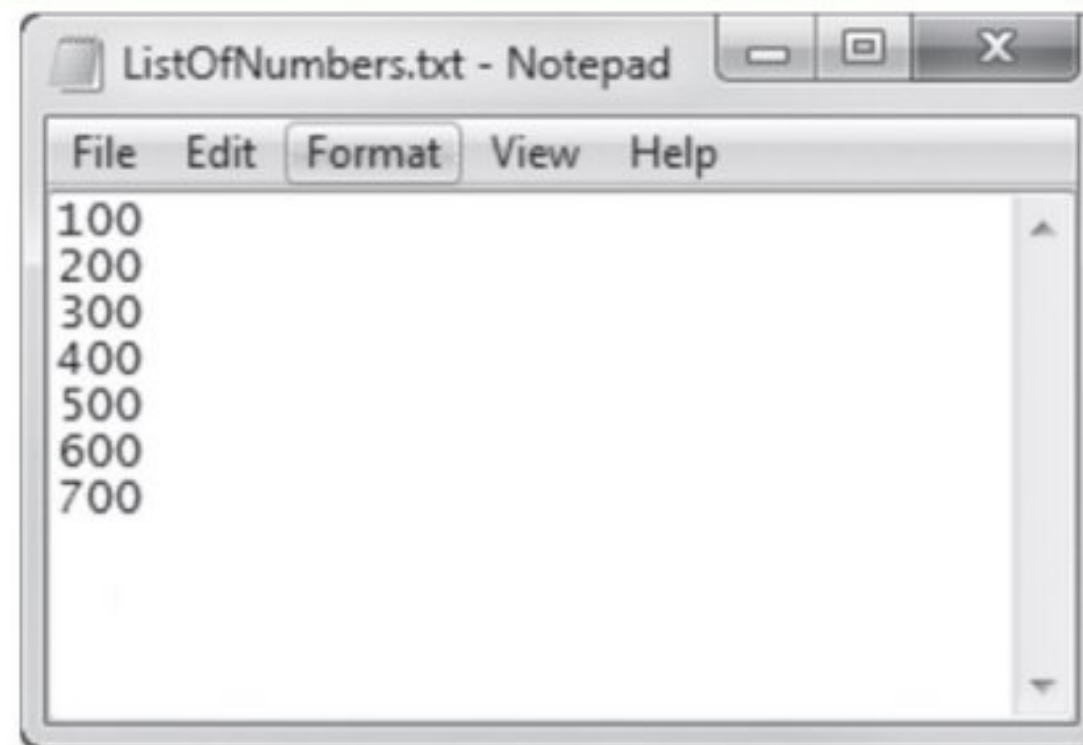
في كثير من الأحيان، يتعين على البرنامج قراءة محتويات ملف دون معرفة عدد العناصر المخزنة فيه. على سبيل المثال، لنفترض أنك بحاجة إلى كتابة برنامج يعرض جميع العناصر في ملف، ولكنك لا تعرف عدد العناصر التي يحتويها.

يمكنك فتح الملف ثم استخدام حلقة تكرارية لقراءة عنصر منه بشكل متكرر وعرضه. ومع ذلك، سيحدث خطأ إذا حاول البرنامج القراءة بعد نهاية الملف.

يحتاج البرنامج إلى طريقة لمعرفة وقت الوصول إلى نهاية الملف حتى لا يحاول القراءة بعد ذلك. **لحسن الحظ، لا يقرأ عامل >> البيانات من الملف فحسب، بل يرجع أيضاً قيمة صحيحة أو خاطئة تُشير إلى ما إذا كانت البيانات قد تمت قراءتها بنجاح أم لا.** إذا أعاد عامل التشغيل القيمة الصحيحة، فهذا يعني أن القيمة قد تمت قراءتها بنجاح. إذا أعاد عامل التشغيل القيمة الخاطئة، فهذا يعني أنه لم تتم قراءة أي قيمة من الملف.

لنلق نظرة على مثال. يحتوي ملف ListOfNumbers.txt، الموضح في الشكل 5-25، على قائمة أرقام. يفتح البرنامج 5-22 الملف، دون معرفة عدد الأرقام التي يحتويها، ويقرأ جميع القيم التي يحتويها ويعرضها.

Figure 5-25 Contents of ListOfNumbers.txt



Program 5-22

```
1 // This program reads data from a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     ifstream inputFile;
9     int number;
10
11     // Open the file.
12     inputFile.open("ListOfNumbers.txt");
13
14     // Read the numbers from the file and
15     // display them.
16     while (inputFile >> number)
17     {
18         cout << number << endl;
19     }
20
21     // Close the file.
22     inputFile.close();
23     return 0;
24 }
```

Program Output

```
100
200
300
400
500
600
700
```

ألق نظرة فاحصة على السطر 16:

while (inputFile >> number)

لاحظ أن العبارة التي تستخرج البيانات من الملف تُستخدم كتعبير منطقي في حلقة while. تعمل كالتالي:

- يُنفَّذ التعبير **inputFile >> number**.
- إذا تمت قراءة عنصر بنجاح من الملف، يُخزَّن العنصر في المتغير **number**، ويُعيد التعبير **true** للإشارة إلى نجاح العملية. في هذه الحالة، تُنفَّذ العبارة في السطر 18 وتُكرَّر الحلقة.
- إذا لم تكن هناك عناصر أخرى لقراءتها من الملف، يُعيد التعبير **inputFile >> number** القيمة **false**، مما يشير إلى عدم قراءة أي قيمة. في هذه الحالة، تنتهي الحلقة.

لأن القيمة المُعادة من المُعامل **>>** تتحكم في الحلقة، فستقرأ العناصر من الملف حتى الوصول إلى نهاية الملف.

Testing for File Open Errors

في ظروف معينة، لن تعمل دالة العضو المفتوحة. على سبيل المثال، سيفشل الكود التالي إذا لم يكن ملف **info.txt** موجودًا:

```
ifstream inputFile;  
inputFile.open("info.txt");
```

هناك طريقة لتحديد ما إذا كانت دالة العضو (**open**) قد فتحت الملف بنجاح. بعد استدعاء دالة العضو (**open**)، يمكنك اختبار كائن تدفق الملف كما لو كان تعبيرًا منطقيًا. يعرض البرنامج 5-23 مثالاً.

Program 5-23

```
1 // This program tests for file open errors.  
2 #include <iostream>  
3 #include <fstream>  
4 using namespace std;  
5  
6 int main()  
7 {  
8     ifstream inputFile;  
9     int number;  
10  
11     // Open the file.  
12     inputFile.open("BadListOfNumbers.txt");  
13  
14     // If the file successfully opened, process it.  
15     if (inputFile)  
16     {  
17         // Read the numbers from the file and  
18         // display them.  
19         while (inputFile >> number)  
20         {  
21             cout << number << endl;  
22         }  
23     }
```

Program 5-23 (continued)

```

24         // Close the file.
25         inputFile.close();
26     }
27     else
28     {
29         // Display an error message.
30         cout << "Error opening the file.\n";
31     }
32     return 0;
33 }

```

Program Output (Assume BadListOfNumbers.txt does not exist)

```
Error opening the file.
```

لنلق نظرة فاحصة على أجزاء مُحددة من الكود. يستدعي السطر 12 دالة العضو `open` لكائن `inputFile` لفتح الملف `ListOfNumbers.txt`. بعد ذلك، تختبر عبارة `if` في السطر 15 قيمة كائن `inputFile` كما لو كانت تعبيراً منطقياً. عند الاختبار بهذه الطريقة، يُعطي كائن `inputFile` قيمة صحيحة إذا تم فتح الملف بنجاح. وإلا، يُعطي قيمة خاطئة. يُظهر مُخرج المثال أن هذا البرنامج سيعرض رسالة خطأ إذا لم يتمكن من فتح الملف.

هناك طريقة أخرى للكشف عن محاولة فتح ملف فاشلة وهي استخدام دالة العضو `fail`، كما هو موضح في الكود التالي:

```

ifstream inputFile;
inputFile.open("customers.txt");
if (inputFile.fail())
{
    cout << "Error opening file.\n";
}
else
{
    // معالجة الملف
}

```

تعيد دالة العضو `fail` القيمة "صحيح" عند فشل محاولة تشغيل ملف.

عند استخدام إدخال/إخراج ملف، يجب عليك دائماً اختبار كائن تدفق الملف للتأكد من فتح الملف بنجاح. في حال تعذر فتح الملف، يجب إبلاغ المستخدم واتخاذ البرنامج للإجراء المناسب.

Letting the User Specify a Filename

في كلِّ من الأمثلة السابقة، يُدرَج اسم الملف المفتوح ضمن البرنامج كحرف سلسلة نصية. في كثير من الحالات، قد ترغب في أن يُحدِّد المستخدم اسم الملف الذي سيفتحه البرنامج. في إصدار C++ 11 والإصدارات الأحدث، يمكنك تمرير كائن سلسلة نصية كمعامل إلى دالة العضو `open` لكائن تدفق الملفات. يُظهر البرنامج 5-24 مثالاً.

هذه نسخة مُعدَّلة من البرنامج 5-23. تُطالب هذه النسخة المستخدم بإدخال اسم الملف. في السطر 15، يُخزَّن الاسم الذي يُدخله المستخدم في كائن سلسلة نصية يُسمى `filename`. في السطر 18، يُمرَّر كائن `filename` كمعامل إلى دالة `open`.

Program 5-24

```
1 // This program lets the user enter a filename.
2 #include <iostream>
3 #include <string>
4 #include <fstream>
5 using namespace std;
6
7 int main()
8 {
9     ifstream inputFile;
10    string filename;
11    int number;
12
13    // Get the filename from the user.
14    cout << "Enter the filename: ";
15    cin >> filename;
16
17    // Open the file.
18    inputFile.open(filename);
19
```

```

20 // If the file successfully opened, process it.
21 if (inputFile)
22 {
23     // Read the numbers from the file and
24     // display them.
25     while (inputFile >> number)
26     {
27         cout << number << endl;
28     }
29
30     // Close the file.
31     inputFile.close();
32 }
33 else
34 {
35     // Display an error message.
36     cout << "Error opening the file.\n";
37 }
38 return 0;
39 }

```

Program Output with Example Input Shown in Bold

```

Enter the filename: ListOfNumbers.txt 
100
200
300
400
500
600
700

```

5.12 Optional Topics: Breaking and Continuing a Loop

المفهوم: تؤدي عبارة break إلى إنهاء الحلقة مبكرًا. تؤدي عبارة continue إلى إيقاف تكرارها الحالي وبدء التكرار التالي.

تحذير! استخدم عبارتي break وcontinue بحذر شديد. نظرًا لتجاوزهما الشرط الاعتيادي الذي يتحكم في تكرارات الحلقة، فإن هذه العبارتين تُصعِّبان فهم الكود وتصحيحه. لهذا السبب، يُنصح بتجنب استخدام عبارتي break وcontinue قدر الإمكان. مع ذلك، ولأنهما جزء من لغة C++، فسوف نناقشهما بإيجاز في هذا القسم.

أحيانًا يكون من الضروري إيقاف حلقة برمجية قبل إكمال جميع تكراراتها. يمكن أيضًا وضع عبارة `break`، التي استُخدمت مع الدالة `switch` في المحاضرة الرابعة، داخل حلقة برمجية.

عند مواجهتها، تتوقف الحلقة، وينتقل البرنامج إلى العبارة التي تلي الحلقة مباشرةً.

يبدو أن حلقة `while` في مقطع البرنامج التالي تُنفذ عشر مرات، لكن عبارة `break` تُوقفها بعد التكرار الخامس.

```
int count = 0;
while (count++ < 10)
{
    cout << count << endl;
    if (count == 5)
        break;
}
```

Using break in a Nested Loop

في حلقة برمجية متداخلة، تُقاطع عبارة `break` الحلقة التي وُضعت فيها فقط. يعرض مقطع البرنامج التالي خمسة صفوف من النجوم على الشاشة. تتحكم الحلقة الخارجية في عدد الصفوف، بينما تتحكم الحلقة الداخلية في عدد النجوم في كل صف. صُممت الحلقة الداخلية لعرض 20 نجمة، لكن عبارة `break` تُوقفها خلال التكرار الحادي عشر.

```
for (int row = 0; row < 5; row++)
{
    for (int star = 0; star < 20; star++)
    {
        cout << '*';
        if (star == 10)
            break;
    }
    cout << endl;
}
```

The output of the program segment above is:

```
*****
*****
*****
*****
*****
```

The continue Statement

تؤدي عبارة continue إلى إنهاء التكرار الحالي للحلقة فوراً. عند مواجهة continue، يتم تجاهل جميع العبارات في نص الحلقة التي تظهر بعده، وتستعد الحلقة للتكرار التالي.

في حلقة while، يعني هذا أن البرنامج ينتقل إلى تعبير الاختبار في أعلى الحلقة.

وكالمعتاد، إذا كان التعبير لا يزال صحيحاً، يبدأ التكرار التالي. في حلقة do-while، ينتقل البرنامج إلى تعبير الاختبار في أسفل الحلقة، والذي يحدد ما إذا كان التكرار التالي سيبدأ. في حلقة for، تؤدي continue إلى تنفيذ تعبير التحديث، ثم تقييم تعبير الاختبار.

يوضح مقطع البرنامج التالي استخدام continue في حلقة while:

```
int testVal = 0;
while (testVal++ < 10)
{
    if (testVal == 4)
        continue;
    cout << testVal << " ";
}
```

يبدو أن هذه الحلقة تعرض الأعداد الصحيحة من 1 إلى 10. عندما تكون قيمة testVal مساوية لـ 4، فإن عبارة continue تتسبب في تخطي الحلقة لعبارة cout وبدء التكرار التالي. ناتج الحلقة هو:

1 2 3 5 6 7 8 9 10

انتهت المحاضرة الخامسة